Mastering Claude Code

From Zero to Expert — Including 50 Hidden Features Most Developers Don’t Know

Contents

- Mastering Claude Code
  - From Zero to Expert
  - About This Book
  - How to Use This Book
- Part 1: Getting Started
  - Chapter 1: What Is Claude Code & Why It’s Different
  - Chapter 2: Installation & Setup
  - Chapter 3: Your First Real Task
  - Chapter 4: Understanding Permission Modes
- Part 2: Productive Development
  - Chapter 5: CLAUDE.md — Teaching Claude About Your Project
  - Chapter 6: Every Slash Command Explained
  - Chapter 7: Git Workflow Automation
  - Chapter 8: Smart Code Navigation & Editing
- Part 3: Advanced Techniques
  - Chapter 9: The Memory System
  - Chapter 10: Hooks — Automate Everything
  - Chapter 11: MCP Servers — Extend Claude Code Infinitely
  - Chapter 12: IDE Integration
- Part 4: Expert Secrets
  - Chapter 13: Prompt Engineering for Claude Code
  - Chapter 14: Parallel Development with Subagents & Worktrees
  - Chapter 15: Custom Skills & Superpowers
  - Deploy
  - Post-deployment Verification
  - Report

- Part 5: Real-World Mastery
  - Chapter 17: Building a Feature Start-to-Finish
  - Chapter 18: Debugging Like a Pro
  - Chapter 19: Cost Optimization & Efficiency
  - Chapter 20: Your Custom Setup Blueprint

Mastering Claude Code

From Zero to Expert

Including 50 Hidden Features Most Developers Don't Know

About This Book

Claude Code is the most powerful AI coding assistant available today — but most developers barely scratch the surface. They use it to ask questions and edit files, missing the 90% of capabilities that transform it from "helpful chatbot" into an autonomous development partner.

This book takes you from zero to expert in a structured progression. You'll learn not just what Claude Code can do, but how expert developers actually use it — the workflows, the shortcuts, the hidden features, and the prompt patterns that 10x your productivity.

Whether you're installing Claude Code for the first time or you've been using it for months and want to unlock its full potential, this book has something for you.

How to Use This Book

Part 1 (Chapters 1-4) — Start here if you're new. Installation, first tasks, understanding how Claude Code works.

Part 2 (Chapters 5-8) — Core productivity. CLAUDE.md, git workflows, code navigation. Read this even if you're experienced — most developers miss key features here.

Part 3 (Chapters 9-12) — Advanced power. Memory, hooks, MCP servers, IDE integration. This is where you graduate from "user" to "power user."

Part 4 (Chapters 13-16) — Expert secrets. Prompt engineering, parallel development, custom skills, and the crown jewel: 50 hidden tips most developers never discover.

Part 5 (Chapters 17-20) — Real-world mastery. Complete case studies, debugging workflows, cost optimization, and building your custom setup.

Convention: Code you should type/run appears in dark code blocks. Tips appear in orange-bordered callout boxes.

Part 1: Getting Started

Chapter 1: What Is Claude Code & Why It's Different

What Claude Code Is

Claude Code is a CLI-based AI coding assistant built by Anthropic. It runs directly in your terminal, and that distinction matters more than you might think.

Unlike web-based AI chatbots where you copy code snippets back and forth, Claude Code operates inside your development environment. It has direct access to your filesystem, your git history, your shell, and your entire project structure. When you ask it to fix a bug, it reads the actual files, makes surgical edits, runs the tests, and commits the fix — all without you touching a single file.

Think of it this way: traditional AI coding tools are like texting a contractor photos of your broken sink. Claude Code is the contractor standing in your kitchen with a wrench.

You launch it with a single command:

```
claude
```

From there, you're in a conversation with an AI that can see everything in your project directory, run any command your shell can run, and modify any file with precision. It's not generating code for you to paste — it's working alongside you, directly in your codebase.

How It Differs from Other AI Tools

The AI coding tool landscape is crowded. Here's how Claude Code compares to what you might already be using:

Feature	ChatGPT / Claude.ai	GitHub Copilot	Cursor / Windsurf	Aider / Continue	Claude Code
Interface	Web browser	Editor extension	Custom editor	Terminal	Terminal / Desktop / IDE
Sees your codebase	No (paste only)	Current file + neighbors	Full project	Full project	Full project
Edits files directly	No	Inline suggestions	Yes	Yes	Yes
Runs shell commands	No	No	Limited	Yes	Full shell access
Git integration	No	No	Basic	Yes	Deep (commit, PR, branch)
Memory across sessions	Limited	No	Limited	No	Yes (CLAUDE.md + memory)
Custom automation	No	No	No	No	Hooks system
Extensible	Plugins	No	MCP support	No	MCP servers + skills
Sub-agents	No	No	No	No	Yes (parallel work)
Works with any editor	N/A	VS Code, JetBrains	Own editor only	VS Code, Neovim	Any editor + terminal

The key differentiators are depth and autonomy. Copilot completes your current line. Cursor edits your current file. Claude Code handles multi-file refactors, runs your test suite, debugs failures, creates pull requests, and remembers what you told it last week.

What It Can Actually Do

Here’s a concrete breakdown of Claude Code’s capabilities:

Capability	What It Means	Example
Read files	Opens any file in your project, displays with line numbers	“Show me the auth middleware”
Edit files	Surgical find-and-replace edits (not rewriting entire files)	“Add error handling to the login function”
Write files	Creates new files from scratch	“Create a migration for the users table”
Run commands	Executes any shell command, captures output	“Run the tests” / “Build the Docker image”
Search by filename	Glob patterns across the entire project	“Find all TypeScript test files”
Search by content	Regex search through file contents	“Find everywhere we call the payments API”
Git operations	Commit, branch, diff, log, create PRs via `gh` CLI	“Commit these changes with a descriptive message”
Read images	Analyzes screenshots, diagrams, UI mockups	“Here’s a screenshot of the bug”
Read PDFs	Extracts and analyzes PDF content	“Summarize this API spec document”
Read notebooks	Understands Jupyter notebooks with outputs	“What does this data analysis notebook do?”
Web fetch	Downloads and reads web pages	“Read the docs at this URL”
Sub-agents	Spawns parallel workers for independent tasks	“Update all 12 API endpoints to the new format”

Capability	What It Means	Example
Persistent memory	Remembers context across conversations	CLAUDE.md files + memory system
Custom hooks	Automated actions triggered by events	Auto-lint before every commit
MCP servers	Connects to external databases, APIs, services	Query your production database directly

This isn't a theoretical feature list. Every item above is something you'll use in your daily workflow once you know how.

The Mental Model

The right mental model for Claude Code is this: **a senior developer sitting next to you who has already read your entire codebase.**

This developer can:

- Instantly recall any file, function, or pattern in your project
- Run any command on your machine
- Make changes across multiple files in a single coordinated operation
- Remember conversations and decisions from previous sessions
- Work on multiple independent tasks simultaneously (via sub-agents)
- Follow your team's conventions automatically (via CLAUDE.md configuration)

But like any developer — even a brilliant one — Claude Code needs your guidance on **what** to build and **why**. It doesn't know your product roadmap. It doesn't know which trade-offs your team prefers. It doesn't know the business context behind a feature request.

The developers who get the most out of Claude Code are the ones who provide clear intent and let it handle the implementation details. Instead of dictating every line of code, they describe the outcome they want and let Claude Code figure out the approach.

Bad prompt: "Add a try-catch block around line 47 of server.js"

Good prompt: "The server crashes when the database is unavailable. Make it handle connection failures gracefully and return a 503 status."

The first tells Claude Code exactly what to type. The second tells it what problem to solve — and it will often find a better solution than you had in mind.

Available Platforms

Claude Code is available on multiple platforms, each with its own strengths:

Platform	Best For	Notes
CLI (Terminal)	Power users, server environments	Most capable, full feature set
Desktop App	Mac and Windows daily development	Native experience, same features as CLI
VS Code Extension	Developers who live in VS Code	Integrated panel, sees editor context
JetBrains Plugin	IntelliJ, PyCharm, WebStorm users	IDE integration with Claude Code features

The CLI is where experts live. It's the most flexible, works over SSH, integrates with any editor via terminal splits, and is always the first to receive new features. This book focuses primarily on the CLI, but nearly everything applies across platforms.

What This Book Covers

Over the next 19 chapters, you'll go from installation to expertise. You'll learn the fundamentals, then the workflows that make experienced developers productive, then the advanced features that most users never discover. By the end, you'll have a Claude Code setup that's customized to your workflow, your team, and your projects — and you'll know exactly how to get the most out of every conversation.

Let's start by getting Claude Code installed and running on your machine.

Chapter 2: Installation & Setup

Getting Claude Code running takes about five minutes. This chapter covers every installation method, authentication option, and configuration detail so you start with a solid foundation.

Prerequisites

Before installing Claude Code, you need two things:

1. **Node.js 18 or later** — Node.js 24 LTS is recommended. Check your version:

```
node --version
# v24.0.0 or higher
```

If you need to install or update Node.js:

```
# macOS (Homebrew)
brew install node

# Or use a version manager (recommended)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.0/install.sh | bash
nvm install 24
nvm use 24
```

2. **An Anthropic account** — Either a Claude Pro/Max subscription or an API key with credits. More on authentication below.

Installation Methods

Claude Code offers several installation paths. Pick the one that fits your workflow.

NATIVE INSTALLER (RECOMMENDED)

The native installer is the fastest path and includes automatic updates:

```
curl -fsSL https://claude.ai/install.sh | bash
```

This installs the `claude` binary to your system PATH and sets up auto-updates so you always have the latest version.

HOMEBREW (MACOS)

```
brew install --cask claude-code
```

This installs the Desktop app, which also makes the `claude` CLI available in your terminal.

NPM (CROSS-PLATFORM)

If you prefer managing it through npm:

```
npm install -g @anthropic-ai/claude-code
```

> **TIP:**
> The npm method doesn't auto-update. You'll need to run `npm update -g @anthropic-ai/claude-code` periodically. The native installer handles updates automatically.

VERIFY INSTALLATION

Regardless of which method you used:

```
claude --version
```

You should see a version string like `Claude Code v1.x.x`. If you get "command not found," check that your PATH includes the installation directory.

Authentication

Claude Code supports two authentication methods.

OPTION 1: CLAUDE PRO/MAX SUBSCRIPTION (EASIEST)

If you have a Claude Pro ($20/month) or Claude Max ($100/month or $200/month) subscription, authentication is seamless:

```
claude
```

On first launch, Claude Code opens your browser for OAuth authentication. Sign in with your Anthropic account, authorize Claude Code, and you're done. Your session persists across terminal restarts.

This is the simplest option and what most individual developers use.

OPTION 2: API KEY

For teams, CI/CD pipelines, or if you prefer pay-per-use:

```
# Set your API key as an environment variable
export ANTHROPIC_API_KEY=sk-ant-api03-...

# Or add it to your shell profile for persistence
echo 'export ANTHROPIC_API_KEY=sk-ant-api03-...' >> ~/.zshrc
source ~/.zshrc
```

Then launch Claude Code normally:

```
claude
```

It will detect the API key automatically. No browser auth needed.

> **TIP:**
> Never commit your API key to git. Add it to your shell profile (`~/.zshrc` or `~/.bashrc`) or use a secrets manager. If you suspect a key has been exposed, rotate it immediately at console.anthropic.com.

DIAGNOSING AUTH ISSUES

If something isn't working, Claude Code has a built-in diagnostic tool:

```
claude doctor
```

This checks your authentication status, Node.js version, network connectivity, and configuration. It's the first thing to run when something seems wrong.

First Launch

Navigate to any project directory and start Claude Code:

```
cd ~/projects/my-app
```

```
claude
```

On your first launch, you'll see:

1. **A welcome message** explaining what Claude Code can do
2. **A permission prompt** asking what level of autonomy to grant (we cover this in detail in Chapter 4)
3. **The conversation interface** — a prompt where you type your requests

Try something simple to verify everything works:

```
What files are in this project? Give me a high-level overview.
```

Claude Code will use its Glob and Read tools to scan your project and give you a structured summary. If you see it reading files and providing an accurate overview, you're all set.

Configuration Basics

Claude Code's configuration lives in two locations:

GLOBAL CONFIGURATION: `~/.CLAUDE/`

This directory stores settings that apply to all your projects:

```
~/.claude/
├── CLAUDE.md           # Global instructions Claude always follows
├── settings.json       # Permissions, preferences, defaults
└── keybindings.json    # Custom keyboard shortcuts
```

The global `CLAUDE.md` is where you put instructions that should apply everywhere — your preferred coding style, your name, common conventions. For example:

```
# Global Rules
- Always use TypeScript strict mode
- Prefer functional components in React
- Use conventional commits format
- My name is Alex — use it in co-author tags
```

PROJECT CONFIGURATION: `.CLAUDE/` AND `CLAUDE.MD`

Each project can have its own configuration:

```
my-project/
├── .claude/
│   ├── settings.json        # Project-level permissions (commit this)
│   └── settings.local.json  # Personal overrides (gitignored)
├── CLAUDE.md                # Project instructions (commit this)
└── ...
```

The project `CLAUDE.md` is one of the most powerful features in Claude Code. It's a markdown file at your project root that Claude reads at the start of every conversation. Your team can commit it to the repo so that everyone — including Claude — follows the same conventions.

```
# Project: My SaaS App

## Stack
- Next.js 15 with App Router
- PostgreSQL via Drizzle ORM
- Tailwind CSS + shadcn/ui

## Conventions
- All API routes go in app/api/
- Use server actions for mutations
- Tests go next to the file they test (*.test.ts)
- Run `npm test` before committing

## Do NOT
- Modify the database schema without discussing first
- Use any CSS framework other than Tailwind
- Add new dependencies without approval
```

We cover CLAUDE.md in full depth in Chapter 5. For now, just know it exists and that it's how you teach Claude Code about your project.

Model Selection

Claude Code gives you access to multiple models with different speed/capability tradeoffs:

Model	Strengths	Best For
Claude Opus 4	Most capable, best reasoning	Complex architecture, multi-file refactors, debugging
Claude Sonnet 4	Fast, highly capable	Daily development, code generation, most tasks
Claude Haiku	Fastest, most affordable	Quick questions, simple edits, file lookups

SWITCHING MODELS

There are several ways to change your active model:

```
# Start with a specific model
claude --model sonnet

# Inside a conversation, use Shift+Tab to cycle models
# Opus → Sonnet → Haiku → Opus

# Or use the slash command
/model sonnet
```

THE /FAST TOGGLE

The `/fast` command is a quick way to optimize for speed within your current model tier:

```
/fast
```

This enables extended thinking optimizations that make responses faster while using the same model. Toggle it again to turn it off.

WHICH MODEL WHEN?

A practical rule of thumb:

- **Starting a new feature** from scratch with complex requirements? Use **Opus**.
- **Everyday coding** — fixing bugs, writing tests, adding endpoints? **Sonnet** handles it well and responds faster.

- **Quick questions** — “What does this function do?” / “Where is this config?”? **Haiku** is instant and costs a fraction.

> **TIP:**
> You can change models mid-conversation. Start with Haiku for exploration and reading, then switch to Opus when you’re ready for Claude to implement something complex. This is a simple way to manage costs without sacrificing quality.

Essential First Commands

Before moving on, try these commands to familiarize yourself with the interface:

Command	What It Does
`/help`	Show all available slash commands
`/status`	Show current model, permissions, and session info
`/cost`	Show token usage and cost for this session
`/clear`	Clear conversation history (start fresh)
`/compact`	Compress conversation to save context window
`Ctrl+C`	Cancel the current operation
`Ctrl+D` or `/exit`	Exit Claude Code

You now have Claude Code installed, authenticated, and configured. In the next chapter, you’ll put it to work on a real coding task.

Chapter 3: Your First Real Task

Theory is useful. Practice is better. In this chapter, you’ll walk through a complete real-world task with Claude Code — from asking questions about an unfamiliar codebase to implementing a feature, running

tests, and committing the result.

Setting Up

Open your terminal and navigate to a project. If you don't have one handy, clone any open-source project to follow along:

```
git clone https://github.com/expressjs/express.git
cd express
claude
```

Claude Code starts and you're in a conversation. Everything you type from here is a request to your AI coding partner.

Asking Questions About the Codebase

The first thing most developers do with Claude Code is understand existing code. This is where it shines — it can read your entire project far faster than you can.

Get a high-level overview:

```
What does this project do? Describe the architecture and main entry points.
```

Claude Code will use Glob to find key files (`package.json`, `README.md`, entry points), then Read to examine them. You'll get a structured summary within seconds.

Trace a specific flow:

```
How does the routing system work? Walk me through what happens when
app.get('/users', handler) is called.
```

Claude Code will search for the relevant source files, read them in order, and explain the flow with references to specific files and line numbers.

Find specific patterns:

```
Find all places where error handling middleware is used.
```

Claude Code uses Grep (its content search tool) with regex patterns to locate every match across the codebase. The results include file paths and line numbers.

> **TIP:**
> You can reference a specific file in your prompt with `@filename`. For example: `Explain what @src/router/index.js does` will make Claude Code prioritize reading that file first.

Reading Files

When Claude Code reads a file, it uses its Read tool and displays the content with line numbers. This matters because those line numbers become reference points for edits.

Behind the scenes, when you ask "Show me the main entry point," Claude Code does something like this:

```
Read: /path/to/project/src/index.js (lines 1-50)
```

And you'll see the tool output with numbered lines:

```
 1  const express = require('express');
 2  const app = express();
 3
 4  app.use(express.json());
 5  app.use(express.urlencoded({ extended: true }));
 6
 7  // Routes
 8  app.use('/api/users', require('./routes/users'));
 9  app.use('/api/posts', require('./routes/posts'));
10
11  app.listen(3000, () => {
12    console.log('Server running on port 3000');
13  });
```

For large files, Claude Code reads only the relevant sections rather than loading the entire file. This is intentional — it keeps the conversation context efficient and focused.

Editing a File

Now let's make a change. Say you notice the server has no health check endpoint. Ask Claude Code to add one:

```
Add a health check endpoint at GET /health that returns { status: "ok", uptime: process.uptime
() }
```

Claude Code will:

1. Read the entry point file to understand the structure
2. Use the **Edit tool** to insert the new code at the right location

The Edit tool works by specifying an `old_string` (existing code to find) and a `new_string` (what to replace it with):

```
Edit: src/index.js
  old_string: "// Routes"
  new_string: "// Health check\napp.get('/health', (req, res) => {\n
    res.json({ status: 'ok', uptime: process.uptime() });\n});\n\n// Routes"
```

This is a key concept: **Edit sends only the diff, not the entire file.** This makes it:

- **Cheaper** — fewer tokens consumed
- **Safer** — can't accidentally overwrite unrelated code
- **Precise** — targets exactly the code that needs to change

If the old_string isn't found (perhaps the code structure differs from what Claude expected), the edit fails gracefully and Claude Code re-reads the file to try again with the correct context.

> **TIP:**
> The Edit tool is always preferred over the Write tool for modifications. Write replaces the entire file contents, which risks losing changes. Edit touches only the specific lines that need to change. You'll see Claude Code use Edit for nearly every modification — this is by design.

Running Commands

Claude Code can execute any shell command through its Bash tool. This means building, testing, linting, deploying — anything your terminal can do.

Run tests:

```
Run the test suite and tell me if anything fails.
```

Claude Code will find and execute the appropriate test command (reading `package.json` scripts, `Makefile`, or whatever your project uses):

```
npm test
```

It captures the full output — pass/fail counts, error messages, stack traces — and summarizes the results. If tests fail, it can immediately analyze the failures and propose fixes.

Build the project:

```
Build the project and make sure there are no TypeScript errors.
```

```
npm run build
```

Run arbitrary commands:

```
Check which port 3000 is currently using.
```

```
lsof -i :3000
```

Claude Code isn't limited to project scripts. It has full shell access, so system commands, curl requests, Docker operations — anything goes.

> **TIP:**
> You can run a quick shell command without waiting for Claude Code to process it by prefixing with `!`. For example, `!git status` runs immediately and injects the output into the conversation. This is useful for giving Claude Code context without a full round-trip.

Creating New Files

When you need something that doesn't exist yet, Claude Code uses the Write tool to create files from scratch:

```
Create a middleware function in src/middleware/requestLogger.js that logs
the method, URL, and response time for every request.
```

Claude Code will:

1. Check the project structure to understand conventions (file locations, import style, existing middleware patterns)
2. Create the file with idiomatic code that matches your project's style
3. Often suggest how to wire it into your application

The Write tool creates the file with the complete content in one operation. For new files, this is the right tool. For modifying existing files, Edit is preferred.

Multi-step Tasks

This is where Claude Code truly differentiates itself. Give it a complex task and watch it plan and execute across multiple files:

```
Add a rate limiting middleware that:
- Limits each IP to 100 requests per 15-minute window
- Returns 429 Too Many Requests when exceeded
- Has configurable limits via environment variables
- Include tests
```

Claude Code will typically:

1. **Plan** — Identify which files to create or modify
2. **Create the middleware** — Write the rate limiter with the specified behavior
3. **Create the test file** — Write tests covering normal requests, rate limit exceeded, configuration, and edge cases
4. **Wire it up** — Edit the entry point to use the new middleware
5. **Run the tests** — Execute the test suite to verify everything works
6. **Fix issues** — If tests fail, analyze the errors and fix them

All of this happens in a single conversation. You describe the outcome, Claude Code handles the implementation across as many files and steps as needed.

Conversation Flow Tips

How you interact with Claude Code matters. Here are patterns that experienced users rely on:

GIVE FOLLOW-UP INSTRUCTIONS

After Claude Code completes a task, you can refine it:

```
Good, but use Redis instead of an in-memory Map for the rate limit store.
Also make the key prefix configurable.
```

Claude Code maintains the full conversation context, so it knows exactly what you're referring to and what code it already wrote.

ASK FOR ALTERNATIVES

```
Show me a different approach — what if we used a sliding window
instead of fixed windows?
```

COURSE-CORRECT EARLY

If you see Claude Code heading in a direction you don't want:

Press `Escape` to interrupt the current operation, then:

```
Stop — don't use that library. Implement it with native Node.js only,
no external dependencies.
```

Interrupting is free and doesn't waste your context window. It's better to stop early than let Claude Code finish something you'll reject.

START FRESH FOR NEW TOPICS

Each conversation has a context window — a limit on how much text it can hold. Long conversations with many file reads and command outputs consume context quickly.

When you switch to a completely different task, start a new conversation:

```
/clear
```

Or exit and restart:

```
claude
```

This gives Claude Code a fresh context window, which means better performance and more accurate responses. Your CLAUDE.md instructions are automatically reloaded, so project context is never lost.

> **TIP:**
> Use `/compact` if you're deep in a productive conversation but running low on context. It compresses the conversation history, preserving key decisions and context while freeing up space for more work.

What You Just Learned

In this chapter, you used Claude Code to:

- Explore and understand an unfamiliar codebase
- Read files with precise line numbers
- Make targeted edits without rewriting entire files
- Run shell commands and analyze their output
- Create new files that match project conventions
- Execute multi-step tasks spanning multiple files
- Manage conversation flow effectively

This is the basic loop of working with Claude Code. In the next chapter, we'll look at the permission system that governs how much autonomy Claude Code has — and how to configure it for your comfort level.

Chapter 4: Understanding Permission Modes

Claude Code can read your files, rewrite them, and execute arbitrary shell commands. That's what makes it powerful — and it's exactly why permissions exist. This chapter explains the permission system that lets you control how much autonomy Claude Code has, from "ask me everything" to "handle it all."

Why Permissions Exist

Every time Claude Code wants to do something beyond reading files and searching, it needs your approval — unless you've configured it otherwise. This is a deliberate safety design:

- **File edits** change your source code
- **Shell commands** can install packages, delete files, modify system state
- **Git operations** can push to remote repositories

Permissions let you decide where on the spectrum between "manual approval for everything" and "full autonomy" you want to operate. There's no single right answer — it depends on the task, the project, and your comfort level.

The Permission Modes

Claude Code has four distinct permission modes. You can switch between them at any time.

ASK MODE (DEFAULT)

```
Permission level: Most restrictive
```

In Ask mode, Claude Code requests your approval before every file edit and every shell command. It can freely read files, search with Glob and Grep, and analyze code, but any action that changes state requires a "yes" from you.

This is the default for good reason — it's how you learn what Claude Code does under the hood. You see every tool call, every edit, every command before it executes. After a few sessions in Ask mode, you'll develop intuition for when Claude Code needs more freedom.

When to use Ask mode:

- First week of using Claude Code (learning phase)
- Working in sensitive codebases (production configs, infrastructure)
- When you want to review every change before it happens
- Pair-programming style: you direct, Claude Code proposes, you approve

AUTO-EDIT MODE

```
Permission level: Moderate
```

Auto-edit mode automatically approves file edits but still asks before running shell commands. This is the sweet spot for most daily development work.

The reasoning is practical: file edits are easily reversible with `git checkout` or undo, but shell commands can have side effects that are harder to reverse (installing packages, modifying databases, pushing to remote).

When to use Auto-edit mode:

- Daily feature development
- Refactoring sessions
- Bug fixes where you trust Claude Code's edits but want to control what runs

FULL-AUTO MODE

```
Permission level: Most permissive
```

Full-auto mode lets Claude Code edit files, run commands, and execute multi-step workflows without asking. It will still pause before certain dangerous operations (more on that in the safety section below).

This mode is remarkably productive for well-defined tasks. You describe what you want, press Enter, and come back to find the feature implemented, tested, and ready for review.

```
Full auto: Add pagination to the /api/users endpoint. Include offset
and limit query params, default to 20 items per page, and add tests.
```

In full-auto mode, Claude Code will plan, implement, create tests, run them, fix any failures, and report back — all without interrupting you.

When to use Full-auto mode:

- Executing a plan you've already reviewed
- Repetitive tasks across many files

- Tasks where the scope is well-defined and low-risk
- Prototyping and exploration in a throwaway branch

> **TIP:**
> Full-auto mode pairs well with git branches. Create a branch, switch to full-auto, let Claude Code implement the feature, then review the diff before merging. If you don't like the result, delete the branch. Zero risk.

PLAN MODE

```
Permission level: Read-only
```

Plan mode is the opposite of full-auto. Claude Code can read, search, and analyze your entire codebase, but it cannot modify any files or run any commands. It proposes changes in detail without executing them.

This is the code review and architecture analysis mode. Claude Code will describe exactly what it would change, in which files, and why — but the actual execution is up to you.

When to use Plan mode:

- Code review: "Review the changes in this PR and flag issues"
- Architecture analysis: "How would you restructure the auth system?"
- Learning: "Explain how the caching layer works and what you'd improve"
- Sensitive systems where even file reads should be the only automated action

Allowlists and Denylists

For finer control than the four modes provide, Claude Code supports allowlists and denylists in `settings.json`. These let you pre-approve or permanently block specific tools and commands.

ALLOWING SPECIFIC COMMANDS

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep",
```

```
      "Bash(npm test)",
      "Bash(npm run build)",
      "Bash(npm run lint)",
      "Bash(git status)",
      "Bash(git diff *)"
    ]
  }
}
```

With this configuration, Claude Code can always run tests, build, lint, and check git status without asking — even in Ask mode. Everything else still requires approval.

BLOCKING DANGEROUS COMMANDS

```
{
  "permissions": {
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(DROP TABLE *)",
      "Edit(.env)"
    ]
  }
}
```

Denied tools are blocked regardless of permission mode. Even in full-auto, Claude Code cannot execute denied commands.

PATTERN MATCHING

Allowlists and denylists support glob-style patterns:

Pattern	Matches
`Bash(npm *)`	Any npm command
`Bash(git status)`	Exact match only
`Bash(docker *)`	Any Docker command
`Edit(*.test.ts)`	Edits to test files only
`Edit(src/*)`	Edits within the src directory

This gives you surgical control. You can auto-approve Claude Code to run tests and edit source files while requiring approval for package installation and deployment commands.

Three Layers of Settings

Settings cascade through three layers, from broadest to most specific:

Layer	File Location	Applies To	Share With Team?
Global	`~/.claude/settings.json`	All projects on your machine	No (personal)
Project	`.claude/settings.json`	This repository	Yes (commit it)
Local	`.claude/settings.local.json`	This repo, only you	No (gitignored)

Global settings are your personal defaults. If you always want `npm test` to be auto-approved, put it here.

Project settings are shared with your team. If your project uses a specific test runner or has commands that are always safe, define them here so everyone gets the same experience. This file should be committed to your repository.

Local settings are your personal overrides for a specific project. Maybe you trust full-auto in this repo even though the team default is Ask mode. This file is automatically gitignored.

Settings merge with increasing specificity. A project-level deny overrides a global-level allow. A local-level setting overrides both.

```
Global: allow Bash(npm test)
Project: deny Bash(npm run deploy)
Local: allow Bash(npm run deploy)
→ Result: npm test is allowed, npm run deploy is allowed (local override)
```

When to Use Each Mode — Decision Guide

Situation	Recommended Mode
First week with Claude Code	Ask
Learning a new codebase	Plan or Ask
Daily feature development	Auto-edit
Writing tests for existing code	Auto-edit or Full-auto
Refactoring with confidence	Full-auto (on a branch)
Running a known implementation plan	Full-auto
Rapid prototyping	Full-auto
Working in production infrastructure	Ask
Code review	Plan
Architecture discussion	Plan

Most experienced Claude Code users land on **Auto-edit as default** with specific commands allowlisted (tests, builds, linting). They switch to Full-auto for well-scoped tasks on feature branches and drop to Ask mode when working near sensitive code.

Built-in Safety Rails

Even in the most permissive mode, Claude Code maintains safety boundaries:

- **Never force-pushes** to a remote repository without explicit confirmation
- **Never deletes branches** without asking

- **Warns before committing sensitive files** (`.env`, credentials, API keys)
- **Respects hooks** — the hook system (Chapter 10) acts as an enforcement layer above permissions. A pre-commit hook that runs linting cannot be bypassed, regardless of permission mode
- **Respects deny rules** — denied tools are never executed, period
- **Pauses on destructive operations** — commands that could cause irreversible damage (`rm -rf`, `DROP TABLE`, `git reset --hard`) trigger additional confirmation

These safety rails mean that full-auto mode is significantly less dangerous than it might sound. Claude Code is designed to be aggressive about productivity and conservative about irreversible actions.

Changing Modes Mid-Conversation

You don't have to decide your permission mode at the start of a session. Switch freely as the situation changes:

```
# Check current mode
/permissions

# Switch to auto-edit
/auto-edit

# Switch to plan mode for review
/plan

# Go full auto for implementation
/auto
```

A common workflow: start in Plan mode to discuss an approach, switch to Auto-edit for implementation, then switch to Full-auto for running tests and fixing failures. This gives you maximum control when it matters and maximum speed when it's safe.

> **TIP:**
> If you accidentally approve a command you didn't mean to, `Ctrl+C` cancels it immediately. For file edits, `git diff` shows you exactly what changed, and `git checkout -- filename` reverts it. You always have an undo button.

Practical Configuration Example

Here's a real-world `settings.json` that balances productivity and safety:

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep",
      "Bash(npm test *)",
      "Bash(npm run build)",
      "Bash(npm run lint *)",
      "Bash(npx tsc --noEmit)",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(git branch *)"
    ],
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force *)",
      "Bash(git reset --hard *)",
      "Edit(.env*)",
      "Edit(*secret*)",
      "Edit(*credential*)"
    ]
  }
}
```

This configuration says: “Claude Code can always read, search, test, build, lint, and check git state. It can never force-push, hard-reset, or touch secrets files. Everything else — ask me first.”

That’s a solid foundation. You’ll refine it over time as you learn which commands you always approve and which you always want to review.

Part 2: Productive Development

Chapter 5: CLAUDE.md — Teaching Claude About Your Project

If you only do one thing to improve your Claude Code experience, it's this: create a CLAUDE.md file.

CLAUDE.md is a markdown file that Claude reads automatically at the start of every conversation. It contains your project's rules, conventions, architecture notes, and any context that Claude needs to do its job well. Think of it as `.editorconfig` for AI — except instead of tab width and line endings, you're specifying how your entire project works.

Without a CLAUDE.md, Claude Code starts every conversation as a talented developer who has never seen your project before. With a good CLAUDE.md, it starts as a developer who has been briefed by the team lead on day one. That difference compounds across every interaction.

Three Levels of CLAUDE.md

CLAUDE.md files cascade through three levels, from global (all projects) to project-specific to personal overrides:

Level	Location	Applies To	Commit to Git?
Global	`~/.claude/CLAUDE.md`	Every project on your machine	No (personal)
Project	`CLAUDE.md` (repo root)	This repository, all team members	Yes
Local	`.claude/CLAUDE.md`	This repo, only you	No (gitignored)

Global is your personal baseline. Put your preferred language, coding style, name, and universal habits here. These apply to every project you open with Claude Code.

Project is the team-shared file at the repository root. Commit it to git. When a new team member clones the repo and runs Claude Code, they get the same instructions as everyone else. This is the file that has the most impact — it teaches Claude Code about your specific project.

Local is your personal override for a specific project. Maybe the team uses `npm test` but you prefer running `npm test -- --watch`. Maybe you want Claude to use a different language than the rest of the

team. The local CLAUDE.md lets you customize without affecting anyone else.

All three levels are merged at conversation start, with more specific levels winning on conflicts.

What to Include

A good CLAUDE.md answers the questions Claude Code would ask if it were a new developer joining your team. Here are the categories that matter most:

Project architecture overview — What does this project do? What's the high-level structure?

Tech stack and key libraries — Framework, language version, major dependencies. Claude Code can detect these from `package.json` or `Cargo.toml`, but explicit declarations prevent guesswork.

Build, test, and lint commands — The commands you use daily. Claude Code uses these when running tests or checking if changes compile.

File and folder conventions — Where things live. Where to put new files. Naming patterns.

Coding style rules — Especially rules that linters don't cover. "We prefer early returns." "We use named exports." "All API responses follow this shape."

Common gotchas — Things that trip people up. "Don't import server modules in client components." "The database migration system requires running X first."

"Don't" rules — Explicit prohibitions are surprisingly effective. "Don't add new npm dependencies without asking." "Never modify the auth middleware directly."

Real-World Examples

Here are three CLAUDE.md files at different complexity levels.

SIMPLE PERSONAL PROJECT

```
# My Portfolio Site

Static site built with Astro. Content in src/content/. Styles in
src/styles/ using vanilla CSS.

## Commands
- `npm run dev` — local server on port 4321
- `npm run build` — production build to dist/

## Rules
```

```
- No JavaScript frameworks — this is a static site
- Images go in public/images/
- Keep pages under 100KB total
```

Ten lines. That's it. Even this small file prevents Claude Code from suggesting React components, putting images in the wrong directory, or using a build command that doesn't exist.

TEAM WEB APPLICATION

```
# Acme Dashboard

Next.js 15 App Router + PostgreSQL (Drizzle ORM) + Tailwind CSS + shadcn/ui

## Architecture
- `app/` — Next.js App Router pages and layouts
- `app/api/` — API route handlers
- `lib/` — Shared utilities, database client, auth helpers
- `components/` — React components (shadcn/ui based)
- `drizzle/` — Database schema and migrations

## Commands
- `npm run dev` — development server (port 3000)
- `npm test` — run Vitest suite
- `npm run db:push` — push schema changes to database
- `npm run db:generate` — generate migration files
- `npm run lint` — ESLint + Prettier check

## Conventions
- Server Components by default, "use client" only when needed
- Use server actions for mutations (not API routes)
- All database queries go through lib/db.ts
- Tests live next to their source file: `foo.ts` → `foo.test.ts`
- Use conventional commits: feat:, fix:, chore:, docs:

## Do NOT
- Add npm dependencies without discussing first
- Modify drizzle/schema.ts without creating a migration
- Use inline styles — Tailwind only
- Import from lib/db.ts in client components
```

Thirty lines covering everything a developer needs to work in this codebase. Every rule here prevents a real mistake that an AI (or human) would otherwise make.

COMPREHENSIVE ENTERPRISE PROJECT

Platform API — Microservices Backend

Overview
Multi-service backend for the Acme platform. 12 services in a monorepo,
deployed to Kubernetes via Helm charts. Language: Go 1.22.
API standard: gRPC with REST gateway.

Repository Structure
- `services/` — Individual microservices (each has its own go.mod)
- `proto/` — Protobuf definitions (source of truth for all APIs)
- `pkg/` — Shared Go packages used across services
- `deploy/` — Helm charts and Kubernetes manifests
- `scripts/` — Build, test, and deployment automation
- `docs/` — Architecture Decision Records (ADRs) and API docs

Build & Test
- `make build` — build all services
- `make build-<service>` — build specific service
- `make test` — run all tests
- `make test-<service>` — test specific service
- `make lint` — golangci-lint across all services
- `make proto` — regenerate code from protobuf definitions

Coding Standards
- Follow Uber Go Style Guide
- Error handling: always wrap errors with fmt.Errorf("context: %w", err)
- Logging: use structured logging (slog) — never fmt.Println in production
- Context: pass context.Context as first argument to all functions
- Naming: services use PascalCase, packages use lowercase, files use snake_case
- Tests: table-driven tests required for all business logic

API Conventions
- All new APIs defined in proto/ first, then implement
- REST endpoints auto-generated via grpc-gateway — don't write REST handlers
- Pagination: use cursor-based (not offset), fields: next_cursor, page_size
- Errors: use standard gRPC status codes, include error_details field

Do NOT
- Modify proto/ files without updating the changelog
- Add dependencies to pkg/ without team review (it affects all services)
- Use database/sql directly — use the repository pattern in pkg/database
- Deploy to staging without running `make integration-test`
- Skip code review for changes touching pkg/ or deploy/

Common Gotchas
- The auth service uses a different database (CockroachDB, not PostgreSQL)
- Service-to-service calls go through the mesh — never use direct HTTP
- The CI pipeline runs `make lint` with stricter settings than local
- Protobuf field numbers cannot be reused — ever

Sixty lines, densely packed with institutional knowledge. This file saves hours of onboarding time — for humans and AI alike.

The /init Command

Don't want to write your CLAUDE.md from scratch? Claude Code can generate one for you.

Start Claude Code in your project directory and run:

```
/init
```

Claude Code will scan your project — reading `package.json`, `Cargo.toml`, `go.mod`, folder structure, git history, README files, CI configs — and generate a CLAUDE.md tailored to what it finds. The result is a solid starting point that you can refine.

The generated file usually captures:

- Project type and framework detection
- Build and test commands from package scripts or Makefiles
- Folder structure conventions based on existing layout
- Language and framework versions

It won't capture your team's unwritten rules or gotchas — those are the parts you add yourself. But as a starting point, `/init` saves significant time.

> **TIP:**
> Run `/init` even on projects that already have a CLAUDE.md. Compare the output with your existing file — Claude Code might detect conventions or commands you forgot to document.

Path-Specific Rules

Some rules only apply to certain parts of your codebase. API files need input validation. Test files need specific patterns. Database migrations need extra caution.

Claude Code supports path-specific rules through `.claude/rules/*.md` files. Each file has YAML frontmatter specifying which paths the rules apply to:

```
---
paths:
  - "src/api/**/*.ts"
---
# API Route Rules

- Always validate request body with zod schema
- Return standardized error responses: { error: string, code: number }
- Include rate limiting headers in responses
- Log all 4xx and 5xx responses with request ID
```

Another example for test files:

```
---
paths:
  - "**/*.test.ts"
  - "**/*.spec.ts"
---
# Test Rules

- Use describe/it blocks, not test()
- Each test file must have at least one happy path and one error case
- Mock external services, never make real HTTP calls
- Use factories for test data, not inline object literals
```

These rules are loaded only when Claude Code is working on matching files. They keep your main CLAUDE.md clean while still providing targeted guidance where it matters.

File Imports

As your CLAUDE.md grows, you might want to reference other documentation files rather than duplicating information. Use the `@path` syntax to import context from other files:

```
# My Project

## Architecture
@docs/architecture.md

## API Conventions
@docs/api-conventions.md

## Database Schema
@docs/schema-overview.md
```

When Claude Code reads the CLAUDE.md, it will also load the referenced files into context. This is powerful for projects that already have extensive documentation — you don't need to rewrite it for Claude, just point to it.

Keep in mind that every imported file consumes context window space, so be selective. Import the files Claude Code needs most often, not your entire docs folder.

Pro Tips for Effective CLAUDE.md Files

Keep it concise. Claude Code reads your CLAUDE.md at the start of every conversation. A 500-line CLAUDE.md consumes context window space that could be used for actual work. Aim for 20-60 lines for most projects. If you need more, use path-specific rules or file imports.

Update as your project evolves. Your CLAUDE.md should be a living document. When you add a new convention, change a build command, or discover a gotcha, update the file. Stale instructions cause Claude Code to make outdated decisions.

Include "don't" rules. Negative instructions are surprisingly effective. "Don't use class components" prevents an entire category of wrong suggestions. "Don't modify the auth middleware" protects critical code.

Specify your expertise level. If you're learning a new framework, tell Claude Code: "I'm new to Rust — explain ownership decisions." If you're an expert: "I know React well — skip basic explanations, just write the code." This calibrates the level of detail in Claude's responses.

Use markdown headers for organization. Claude Code parses CLAUDE.md as markdown. Headers create clear sections that are easy to scan and update. Use `##` for major categories and `###` for subcategories.

Test your CLAUDE.md. After writing or updating it, start a new Claude Code conversation and ask it to do something covered by your rules. Verify it follows them. If it doesn't, your instructions might be ambiguous — rewrite them more explicitly.

> **TIP:**
>
> Want to see what Claude Code currently knows about your project? Use the `/memory` command. It shows your CLAUDE.md contents plus any auto-memory Claude has accumulated across sessions. This is useful for auditing what instructions Claude is actually following.

Chapter 6: Every Slash Command Explained

Slash commands are Claude Code's built-in utilities — fast, single-purpose operations you trigger with `/` followed by a keyword. They manage sessions, change settings, control context, and access features that would be clunky to invoke through natural language.

Most developers learn three or four commands and ignore the rest. This chapter documents every slash command so you know exactly what's available and when to use each one.

Session Management

These commands control the lifecycle of your conversations — starting, stopping, resuming, and organizing them.

Command	Description
`/clear`	Clear the current conversation and start fresh. Your CLAUDE.md is reloaded automatically. Use this when switching to a completely different task.
`/compact [focus]`	Compress the conversation history to reclaim context window space. Optionally provide a focus hint: `/compact focus on auth changes` tells Claude what to preserve during compression.
`/rename [name]`	Give the current session a memorable name. `/rename auth-refactor` makes it easy to find and resume later.
`/resume [name]`	Resume a previous named session. Without a name, shows a list of recent sessions to choose from.
`/continue`	Resume the most recent conversation. Equivalent to launching with `claude --continue`.
`/branch [name]`	Fork the conversation at the current point. Creates a new branch of the conversation so you can explore a different approach without losing the current one.
`/rewind`	Undo the last exchange. Rolls back to the previous message checkpoint, removing Claude's last response and your last prompt. Useful when Claude went in the wrong direction.
`/exit` or `/quit`	End the session and exit Claude Code. Same as `Ctrl+D`.

The most underused command here is `/compact`. Long conversations accumulate file contents, command outputs, and discussion history that eat into your context window. When you notice Claude Code starting to miss details or repeat itself, run `/compact` to free up space. You won't lose important context — Claude summarizes the key decisions and code changes before compressing.

TIP:

Add a focus hint to `/compact` when you're deep in a specific area. `/compact focus on database migration changes` ensures the compression preserves details about the exact work you care about, even if it aggressively compresses earlier discussion about unrelated files.

Model & Configuration

These commands control how Claude Code operates — which model it uses, how much reasoning it applies, and its speed/quality tradeoff.

Command	Description
`/model [model]`	Switch to a different model. `/model opus` for maximum capability, `/model sonnet` for speed, `/model haiku` for quick lookups.
`/fast [on|off]`	Toggle fast mode, which optimizes for response speed. Useful when you're doing quick iterations and don't need deep reasoning.
`/effort [level]`	Set the reasoning depth. Accepts `low`, `medium`, `high`, or `max`. Lower effort means faster but shallower responses. Higher effort means Claude thinks longer before responding.
`/config`	Open the settings interface to adjust permissions, preferences, and defaults interactively.

A practical workflow: start your session with `/effort low` for exploration and file reading, then `/effort high` when you're ready for Claude to implement something complex. This minimizes cost and wait time during the discovery phase.

Development

These commands directly support your coding workflow — from initializing project configuration to reviewing changes.

Command	Description
`/init`	Scan the project and generate a CLAUDE.md file with detected conventions, commands, and structure. Covered in depth in Chapter 5.
`/diff`	Open an interactive diff viewer showing all pending file changes Claude has made. Lets you review and accept or reject individual changes.
`/security-review`	Run a security audit on pending changes. Claude analyzes all modifications for potential vulnerabilities — injection risks, exposed secrets, permission issues.
`/batch <instruction>`	Apply changes in parallel across multiple git worktrees. Useful for making the same change across several branches or services simultaneously.

The `/security-review` command is worth running before any commit that touches authentication, authorization, input handling, or data access. It's a free second pair of eyes that catches issues like SQL injection vectors, missing input validation, or accidentally logged secrets.

Context & Memory

These commands give you visibility into Claude Code's context window usage, memory, and costs.

Command	Description
`/memory`	View and edit your CLAUDE.md files and auto-memory. Shows what instructions Claude is currently following and lets you modify them.
`/cost`	Display token usage and estimated costs for the current session. Shows input tokens, output tokens, cache hits, and total spend.
`/context`	Show a visual grid of context window usage. Reveals how much of the context window is consumed by conversation history, file contents, and tool outputs.
`/export [filename]`	Save the entire conversation as a text file. Defaults to a timestamped filename if none is provided. Useful for documentation or sharing.
`/copy [N]`	Copy Claude's last response (or last N responses) to your clipboard. Handy for pasting code snippets into other tools.

The `/cost` command is your budget tracker. After completing a major task, check `/cost` to understand the token economics. Over time, this builds intuition for which types of requests are expensive (large file reads, multi-file refactors) and which are cheap (quick questions, small edits).

TIP:

The `/context` grid is your early warning system for context window exhaustion. When it shows you're above 70% capacity, consider running `/compact` or starting a new conversation. Performance degrades noticeably as you approach the limit.

Advanced

These commands cover specialized features — scheduling, diagnostics, analytics, and cloud-powered capabilities.

Command	Description
`/schedule`	Create recurring tasks that run on a schedule in the cloud. Claude Code connects to a remote agent that executes your instructions on a cron schedule.
`/btw <question>`	Ask a side question without polluting the main conversation context. The answer is shown but the exchange is not added to the conversation history.
`/ultraplan`	Invoke cloud-based deep planning. Claude creates a detailed implementation plan using extended cloud compute, then returns the plan for your review.
`/status`	Show version info, current model, account type, and session details. First thing to check when something seems wrong.
`/doctor`	Run diagnostics on your Claude Code installation. Checks Node.js version, authentication, network connectivity, and configuration.
`/stats`	Display usage statistics — sessions count, messages sent, tools used, and usage streaks.
`/insights`	Generate an analysis report of the current session — what was accomplished, how tokens were spent, and suggestions for efficiency.

The `/btw` command deserves special attention. During a complex implementation, you might suddenly need to check a syntax detail or look up a library API. A normal question would inject that tangent into the main conversation, potentially confusing Claude's focus on the primary task. `/btw what's the syntax for Go's sync.WaitGroup?` gets you the answer without derailing the session.

Keyboard Shortcuts

These work anywhere in the Claude Code interface. Memorize the top five and add more as they become natural.

Shortcut	Action
`Enter`	Send your message
`Shift+Enter`	Insert a new line (multi-line input)
`Shift+Tab`	Cycle through available models (Opus → Sonnet → Haiku)
`Esc`	Cancel the current generation — Claude stops immediately
`Ctrl+C`	Exit Claude Code (or cancel current operation)
`Ctrl+R`	Search through command history
`Ctrl+O`	Toggle verbose transcript (see raw tool calls and outputs)
`Ctrl+B`	Background the current task — Claude continues working while you get your prompt back
`Ctrl+G`	Open the current conversation in an external editor for review
`Up arrow`	Recall your previous message (edit and resend)

The most important shortcut is `Esc`. The moment you see Claude heading in a wrong direction — wrong file, wrong approach, wrong library — press `Esc`. It costs nothing to interrupt and redirect. Letting Claude finish a bad approach wastes tokens and context window.

`Ctrl+B` is a power-user feature. When you give Claude a large task (refactoring a module, writing a test suite), press `Ctrl+B` to background it. You get your prompt back immediately and can ask other questions or start a new task. Claude notifies you when the backgrounded work is complete.

TIP:

`Shift+Tab` for model cycling is one of the fastest ways to save money. Reading files and exploring? Shift+Tab to Haiku. Ready to implement? Shift+Tab to Opus. Two keystrokes, significant cost difference.

The `!` Prefix — Inline Shell Commands

Any message starting with `!` is executed as a shell command, and the output is injected into the conversation. This is different from asking Claude to run a command — it runs immediately, with no AI processing.

```
! git log --oneline -5
```

Output appears instantly:

```
a1b2c3d feat: add user authentication
e4f5g6h fix: resolve database connection timeout
i7j8k9l chore: update dependencies
m0n1o2p feat: add rate limiting middleware
q3r4s5t docs: update API documentation
```

This output is now part of the conversation context. You can follow up with:

```
Explain what changed in the rate limiting commit (m0n1o2p).
```

Claude Code sees the git log output and uses it to understand your question.

Common uses for the `!` prefix:

Command	Purpose
`! git status`	Show Claude the current state of your working tree
`! git diff`	Feed Claude the exact changes you've made
`! cat .env.example`	Show Claude environment variables without risk
`! npm test 2>&1 | tail -20`	Share the last 20 lines of test output
`! ls -la src/`	Show Claude a directory listing for context

The key advantage is speed. `! git status` runs in milliseconds and injects the result. Asking Claude "run git status" goes through a full AI round-trip — Claude reads the request, decides to use the Bash tool, runs the command, reads the output, and summarizes it. When you just need raw output in the conversation, `!` is faster.

The `@` Reference — File Mentions

Typing `@` followed by a file path loads that file into Claude's context. This is the fastest way to bring a specific file into the conversation without asking Claude to "read" it.

```
@src/middleware/auth.ts explain what this middleware does
```

Claude Code loads the file contents immediately, so it can analyze the code without a separate Read tool call. This is especially useful when you know exactly which file you want to discuss.

You can reference multiple files:

```
@src/models/user.ts @src/routes/users.ts
Are these two files consistent in how they handle user validation?
```

The `@` reference works with relative paths from your project root. Tab completion helps you find the right file — start typing `@src/` and press Tab to see available options.

TIP:

Combine `!` and `@` for maximum context efficiency. Run `! git diff --name-only` to see which files changed, then `@path/to/changed/file.ts` to pull in the specific file you want Claude to focus on. This gives Claude precise context without reading everything.

Chapter 7: Git Workflow Automation

Git is the backbone of every development workflow, and Claude Code is deeply integrated with it. Not just "can run git commands" integrated — Claude Code understands git semantics, follows safety protocols, and handles the entire commit-branch-PR lifecycle with minimal input from you.

This chapter covers every git capability, from one-word commits to complex merge conflict resolution.

Smart Commits

The simplest and most common git operation: committing your changes. Tell Claude Code:

```
commit this
```

Or use the slash command:

```
/commit
```

What happens next is more sophisticated than it appears. Claude Code:

1. **Runs** `git status` to see all untracked and modified files
2. **Runs** `git diff` to see exactly what changed (both staged and unstaged)
3. **Reads recent commit messages** (`git log`) to match your repository's commit style — if you use conventional commits, it follows that; if you use sentence-case, it follows that
4. **Analyzes the actual changes** — not just file names, but the code modifications — to understand what was done and why
5. **Generates a commit message** that focuses on the "why" rather than the "what"
6. **Stages the relevant files** — and this is key — it avoids staging files that shouldn't be committed: `.env`, credentials, large binaries, unrelated changes

7. **Creates the commit** using a HEREDOC format for clean multi-line messages

The result is a commit message that reads like a thoughtful developer wrote it, because one did.

```
feat: add cursor-based pagination to users endpoint

Replace offset pagination with cursor-based approach for better
performance on large datasets. Includes backward compatibility
for clients still sending offset/limit params.
```

Compare this to what most developers write when they're tired and just want to push:

```
update users endpoint
```

> **TIP:**
>
> Claude Code always creates a **new** commit. It never amends an existing commit unless you explicitly say "amend the last commit." This is a safety protocol — amending can overwrite previous work, especially after a failed pre-commit hook where the commit didn't actually happen. If you need to amend, be explicit: "amend the previous commit with these changes."

SELECTIVE STAGING

Claude Code is smart about what it stages. If you've been working on two separate features and only want to commit one:

```
commit just the authentication changes, not the UI updates
```

Claude Code will `git diff` everything, identify which files relate to authentication, stage only those, and write a commit message specific to what's being committed. The UI changes remain in your working tree, ready for a separate commit.

COMMIT WITH INSTRUCTION

You can guide the commit message:

```
commit this as a bug fix — the issue was that passwords weren't being hashed before storage
```

Claude Code will incorporate your context into the message:

```
fix: hash passwords before database storage

Passwords were being stored in plaintext. Add bcrypt hashing in the
user registration and password update flows.
```

Pull Request Creation

Creating a pull request is one of Claude Code's strongest git features. Tell it:

```
create a PR
```

Claude Code performs a thorough analysis before creating anything:

1. **Checks the current branch** and identifies the base branch
2. **Reads ALL commits** since the branch diverged — not just the latest commit, but the entire branch history. This is critical for accurate PR descriptions.
3. **Runs** `git diff base...HEAD` to see the complete set of changes
4. **Checks if the branch is pushed** to remote — if not, it pushes with `-u` flag
5. **Generates a title** (under 70 characters) and a structured description

The PR is created using `gh pr create` with a clean format:

```
## Summary
- Add cursor-based pagination to /api/users endpoint
- Include backward compatibility for offset/limit params
- Add integration tests for pagination edge cases

## Test plan
- [ ] Verify cursor pagination returns correct pages
- [ ] Verify offset/limit still works (backward compat)
- [ ] Test with empty result set
- [ ] Test with single-item result set
- [ ] Load test with 100K+ users
```

The title stays short and descriptive. The body uses the Summary + Test plan format that reviewers appreciate. Details go in the body, not the title.

PR WITH CONTEXT

Add context to guide the PR description:

```
create a PR — this closes issue #42 and was requested by the
frontend team for infinite scroll support
```

Claude Code will reference the issue, add context from your instruction, and link the PR to the issue.

DRAFT PRS

```
create a draft PR — still need to add error handling
```

Claude Code adds the `--draft` flag and notes the remaining work in the description.

Branch Management

Claude Code handles branch operations naturally:

```
create a branch for the user authentication feature
```

```
git checkout -b feature/user-authentication
```

Claude Code picks conventional branch names based on your repository's existing patterns. If your branches use `feat/`, it follows that. If they use `feature/`, it follows that instead.

```
switch to the main branch
```

```
git checkout main
```

```
show me all branches related to payments
```

```
git branch -a | grep payment
```

One crucial safety protocol: **Claude Code never force-pushes or deletes branches without explicit confirmation.** If you say “clean up old branches,” it will list the branches it would delete and ask for your approval before executing.

Conflict Resolution

Merge conflicts are where most developers lose time. Claude Code handles them by understanding both sides of the conflict, not just blindly picking one.

When you run into a conflict:

```
I just merged main into my branch and got conflicts. Fix them.
```

Claude Code will:

1. Run `git diff --name-only --diff-filter=U` to find conflicted files
2. Read each conflicted file, analyzing the conflict markers
3. Understand the intent of both sides — what your branch changed and what main changed
4. Resolve the conflict by merging both intents when possible, or choosing the correct side when they’re incompatible
5. Stage the resolved files

Here’s what this looks like in practice. Say you have this conflict in `src/config.ts`:

```
<<<<<<< HEAD
const MAX_RETRIES = 5;
const TIMEOUT_MS = 10000;
=======
const MAX_RETRIES = 3;
const TIMEOUT_MS = 30000;
const BACKOFF_MULTIPLIER = 2;
>>>>>>> main
```

Claude Code sees that your branch increased `MAX_RETRIES` while main increased `TIMEOUT_MS` and added `BACKOFF_MULTIPLIER`. Instead of picking one side, it merges both intents:

```
const MAX_RETRIES = 5;         // from your branch
const TIMEOUT_MS = 30000;      // from main
const BACKOFF_MULTIPLIER = 2;  // new addition from main
```

It keeps your retry increase, keeps main's timeout increase, and keeps the new constant. If the changes truly conflict (both sides changed the same value for different reasons), Claude Code explains the conflict and asks which side you prefer.

> **TIP:**
>
> For complex conflicts involving many files, give Claude Code context about what you were working on: "My branch adds OAuth support. Main branch refactored the user model. Resolve conflicts preserving both changes." The more intent you provide, the better the resolution.

Git Safety Protocols

Claude Code has built-in safety rails for git operations. These are not configurable — they're always active, even in full-auto mode.

Protection	What It Does
No force push to main/master	Claude Code will warn and refuse to `git push --force` to protected branches. If you insist, it confirms explicitly.
No hard reset without asking	`git reset --hard` destroys uncommitted work. Claude Code always asks first.
No hook bypass	Claude Code never uses `--no-verify` to skip pre-commit hooks. If a hook fails, it fixes the issue and retries.
New commits, not amends	After a pre-commit hook failure, Claude Code creates a new commit rather than amending. Amending after a failed hook would modify the previous commit — not the failed one.
Sensitive file warnings	Before committing `.env`, credentials, API keys, or files matching `*secret*`, Claude Code warns you and asks for confirmation.
No branch deletion	Claude Code never deletes branches without listing them and getting your explicit approval.

These protocols exist because git mistakes are among the most painful in software development. A force-push to main can overwrite a team's work. A hard reset can destroy hours of uncommitted changes. Claude Code is designed to be aggressive about productivity but conservative about irreversible actions.

Advanced Git Operations

Beyond the basics, Claude Code handles the full range of git operations:

Cherry-picking:

```
cherry-pick commit a1b2c3d from the feature/payments branch
```

Claude Code runs the cherry-pick, handles any conflicts using the same conflict resolution logic, and verifies the result compiles.

Interactive rebasing:

```
rebase my last 3 commits to clean up the history — squash the
fixup commits into the main feature commit
```

Claude Code restructures the commits as requested, combining related changes into clean, logical commits.

Bisecting:

```
the login page broke sometime in the last week. help me bisect
to find which commit caused it.
```

Claude Code sets up `git bisect`, runs the relevant tests at each step, and narrows down to the exact commit that introduced the regression.

Stashing:

```
stash my current changes, switch to main, pull, then come back
and reapply my stash
```

Claude Code handles the full stash-switch-pull-switch-apply workflow in sequence.

STARTING FROM A PR

You can launch Claude Code in the context of an existing pull request:

```
claude --from-pr 123
```

This loads the PR description, the diff, and any review comments into the conversation. It's useful for addressing PR feedback — Claude Code has full context of what the reviewer asked and what code they're referring to.

ISOLATED WORKTREES

For parallel work on multiple features:

```
claude --worktree feature-name
```

This creates an isolated git worktree and starts Claude Code inside it. Your main working directory is untouched. When the feature is complete, the worktree is cleaned up. This pairs well with `/batch` for making coordinated changes across multiple branches.

> **TIP:**
> The combination of `--worktree` and full-auto mode is extremely powerful. Create a worktree, give Claude Code a well-defined task in full-auto mode, and let it work independently while you continue in your main branch. Review the result when it's done.

Chapter 8: Smart Code Navigation & Editing

When you ask Claude Code to find a function, read a file, or rename a variable, it doesn't shell out to `grep` and `sed` like a script would. It uses a set of purpose-built tools that are faster, safer, and more context-aware than their shell equivalents.

Understanding these tools — what they do, when they're used, and how to guide their usage — is the difference between Claude Code stumbling through your codebase and navigating it surgically.

The Tool Arsenal

Claude Code has five core tools for interacting with your files. Each one replaces a common shell command with something better:

Instead of...	Claude Uses...	Why It's Better
`find . -name "*.ts"`	**Glob**	Faster pattern matching, results sorted by modification time
`grep -r "pattern" .`	**Grep**	Proper permissions, full regex, multiple output modes, file type filtering
`cat file.py`	**Read**	Line numbers, offset/limit for large files, reads images and PDFs
`sed -i 's/old/new/'`	**Edit**	Exact string match (not regex), preserves formatting, must read first
`echo "content" > file`	**Write**	Creates new files cleanly, requires prior Read for existing files

You rarely invoke these tools directly — you describe what you want and Claude Code picks the right tool. But knowing how they work lets you write better prompts and understand what Claude is doing under the hood.

Glob — File Pattern Matching

Glob finds files by name pattern. It's Claude Code's first move when exploring a project — mapping out the structure before diving into specific files.

Standard glob patterns:

```
**/*.ts           → all TypeScript files, any depth
src/**/*.test.tsx → test files in src/, React + TypeScript
*.config.{js,ts}  → config files, JS or TS, root only
```

Results are sorted by modification time (most recently modified first). This is surprisingly useful — when Claude Code searches for `**/*.ts`, the files you've been working on appear first. This natural prioritization means Claude tends to look at relevant files before stale ones.

Example use cases:

1. **Understanding project structure:** "What files are in this project?" triggers a broad glob (`**/*`) followed by analysis of the directory tree.
2. **Finding related files:** "Find all the migration files" triggers `**/migrations/*.{sql,ts,js}` or similar patterns based on your project's conventions.
3. **Locating config files:** "Where is the database config?" triggers globs for common config patterns (`**/*config*`, `**/*.config.*`, `**/database.*`).

> **TIP:**
> When you ask Claude to find something and it's not finding it, be more specific about the file type or location. "Find the auth middleware" might glob too broadly. "Find the auth middleware in the src/middleware directory" narrows the search immediately.

Grep — Content Search

Grep searches the contents of files — finding where specific code patterns, function calls, variable references, or string literals appear across your codebase.

Claude Code's Grep is built on ripgrep, not GNU grep. This means it's fast (even on massive codebases), respects `.gitignore` automatically, and supports full regex syntax.

Three output modes:

Mode	What It Returns	When Claude Uses It
`files_with_matches`	Just file paths (default)	Finding which files contain a pattern
`content`	Matching lines with context	Reading the actual matching code
`count`	Number of matches per file	Gauging how widespread a pattern is

File filtering: Grep can filter by glob pattern (`*.ts`, `src/**/*.py`) or by file type (`js`, `py`, `rust`). Type filtering is more efficient for standard languages.

Context lines: When using `content` mode, Grep can show surrounding lines — lines before the match (`-B`), after the match (`-A`), or both (`-C`). This helps Claude understand the code around a match without reading the entire file.

Example searches:

1. **Finding all API calls:**

```
Find everywhere we call the Stripe API.
```

Claude Code greps for patterns like `stripe.`, `Stripe(`, `@stripe/` across the codebase, returning file paths and matching lines.

2. **Finding function usage:**

```
Where is the validateEmail function used?
```

Claude greps for `validateEmail` across all files, showing each call site with surrounding context.

3. **Finding patterns across languages:**

Claude Code searches with regex: `function\s+\w+Auth` finds `function checkAuth`, `function requireAuth`, etc. The regex support means complex pattern matching without multiple search passes.

Multiline mode: By default, patterns match within single lines. For patterns that span multiple lines (like finding a function definition with its body), Claude Code enables multiline matching:

```
Find all React components that use both useState and useEffect.
```

This triggers a multiline search for components importing or using both hooks within the same file scope.

Read — Intelligent File Reading

Read opens files and returns their contents with line numbers. It's the most frequently used tool in every Claude Code session.

Line numbers in output: Every Read result looks like `cat -n` output:

```
1  import { Router } from 'express';
2  import { authenticate } from '../middleware/auth';
3
4  const router = Router();
```

```
5
6 router.get('/users', authenticate, async (req, res) => {
7   const users = await db.users.findAll();
8   res.json(users);
9 });
```

These line numbers serve as reference points for discussion. You can say “what happens at line 7?” and Claude knows exactly which line you mean.

Offset and limit for large files: For files with hundreds or thousands of lines, Read supports partial reading:

- “Read lines 100-150 of the config file” uses `offset: 100, limit: 50`
- “Show me the last 30 lines of the log file” uses appropriate offset

This avoids loading massive files into the context window when you only need a section.

Reading images: Claude Code is multimodal. When Read opens a PNG, JPG, or other image format, Claude sees the image visually — not as binary data. This means you can:

```
Look at the screenshot @screenshots/bug-report.png and tell me
what's wrong with the UI.
```

Claude Code reads the image, analyzes it visually, and describes what it sees. This is invaluable for debugging UI issues, reviewing design mockups, or understanding diagram-based documentation.

Reading PDFs: Read handles PDF files with page range support. For large PDFs, specify which pages you need:

```
Read pages 12-15 of the API specification document.
```

Claude Code extracts and presents the content from those specific pages. Maximum 20 pages per request — for larger documents, read in chunks.

Reading Jupyter notebooks: Claude Code reads `.ipynb` files and presents all cells — code, markdown, and outputs — in a coherent format. It can analyze data science notebooks, understand the analysis flow, and suggest improvements.

TIP:

For large files (1000+ lines), always guide Claude to read specific sections. “Read the UserController class in src/controllers/user.ts” is much more efficient than letting Claude read the entire file. “Read lines 200-250 of the config file” is even more precise. This directly impacts your token cost — reading 50 lines costs a fraction of reading 2000.

Edit — Surgical Precision

Edit is Claude Code’s primary tool for modifying existing files. It works by exact string matching — you specify the old text and the new text, and Edit replaces one with the other.

Key characteristics:

1. **Must read first.** Claude Code must Read a file before it can Edit it. This is a safety requirement — it prevents blind modifications to files Claude hasn’t seen. If Claude tries to edit a file it hasn’t read, the tool returns an error.

2. **Exact string match, not regex.** The `old_string` must appear exactly as it exists in the file — same characters, same whitespace, same indentation. This eliminates the class of bugs where a regex accidentally matches more than intended.

3. **Must be unique.** The `old_string` must appear exactly once in the file. If it appears multiple times, the edit fails and Claude Code must provide more surrounding context to make the match unique. This prevents accidental edits to the wrong location.

4. **Preserves formatting.** Edit respects the file’s existing indentation — tabs vs. spaces, indent level, line endings. The replacement text is inserted exactly as specified, without reformatting.

5. **`replace_all` for bulk changes.** When you intentionally want to replace every occurrence (like renaming a variable), Edit supports a `replace_all: true` flag that replaces all matches in the file.

Why Edit over Write? Edit sends only the diff — the old string and the new string. Write sends the entire file contents. For a 500-line file where you’re changing 3 lines, Edit transmits roughly 6 lines of data while Write transmits 500. This makes Edit approximately 10x cheaper in terms of token usage, and significantly safer since it can’t accidentally overwrite unrelated code.

Example edit flow:

You ask: “Add error handling to the database query in the users route.”

Claude Code: 1. **Reads** `src/routes/users.ts` to see the current code 2. **Identifies** the database query that needs error handling 3. **Edits** with:

```
old_string: "const users = await db.users.findAll();\n  res.json(users);"
new_string: "try {\n    const users = await db.users.findAll();\n
  res.json(users);\n  } catch (error) {\n    console.error('Failed to
  fetch users:', error);\n    res.status(500).json({ error: 'Internal
  server error' });\n  }"
```

Only the affected lines are transmitted and modified. Everything else in the file remains untouched.

Multi-File Operations

Claude Code's real strength emerges with operations spanning multiple files. A rename, a refactor, or an API change might touch dozens of files — Claude Code handles this methodically.

Example: Renaming across a codebase

```
Rename the userId field to user_id in all model files and
everywhere it's referenced.
```

Claude Code executes a systematic workflow:

1. **Grep** for `userId` across all files to find every occurrence
2. **Read** each file to understand the context (is it a variable declaration, a function parameter, a database column name?)
3. **Edit** each file, replacing `userId` with `user_id` — using `replace_all: true` when appropriate
4. **Verify** by grepping again to confirm no occurrences were missed
5. **Run tests** (if available) to confirm the rename didn't break anything

This workflow handles the tricky cases that a simple find-and-replace misses: - Import statements that reference `userId` - TypeScript interfaces and type definitions - Database query builder calls - API response objects - Test assertions that check for `userId` in responses

Example: Updating an API response format

```
Change all API endpoints to wrap responses in { data: ..., meta: { timestamp, requestId } }
```

Claude Code finds every API route handler, reads each one to understand the current response format, edits each to add the wrapper, and updates the corresponding tests and type definitions. One prompt,

potentially dozens of coordinated edits.

Navigation Patterns for Large Codebases

When working with unfamiliar or large codebases (thousands of files), experienced Claude Code users follow a consistent pattern:

1. Start broad — map the terrain:

```
What's the overall structure of this project? List the main
directories and their purposes.
```

Claude Code globs the top-level directories, reads key files (README, package.json, main entry points), and builds a mental map.

2. Narrow down — find the relevant area:

```
I need to understand the payment processing flow. Where does
that live?
```

Claude Code greps for payment-related terms, identifies the relevant directories and files, and focuses its attention.

3. Deep dive — understand the specifics:

```
Walk me through the checkout flow from the moment a user
clicks "Buy" to the payment confirmation.
```

Claude Code reads the relevant files in order, tracing the flow from UI handler to API route to service layer to database, explaining each step.

4. Edit — make the change:

```
Add support for Apple Pay to the checkout flow.
```

Now Claude Code has full context — it knows the architecture, the conventions, the existing payment integrations, and the specific files involved. The edits it makes will be consistent with everything it learned in steps 1-3.

Skipping the exploration steps and jumping straight to “add Apple Pay support” forces Claude Code to do all the discovery and implementation in one pass. The result is usually worse — it might put code in the wrong place, miss an integration point, or violate a convention it hadn’t learned yet.

> **TIP:**
> Treat your first few prompts in any session as investment in Claude Code’s context. A minute spent on exploration saves five minutes of correction later. “Explain the auth system before you change it” is almost always worth the extra round-trip.

Watching Claude Work

When Claude Code navigates and edits your code, you can observe its tool calls in real time. Toggling verbose mode with `Ctrl+O` shows every Glob, Grep, Read, and Edit call with their parameters and results.

This is useful for two reasons:

1. **Learning** — You see exactly how Claude Code approaches code navigation. Which patterns does it search for? How does it narrow down from hundreds of files to the one that matters? These strategies are useful for your own development.

2. **Debugging** — If Claude Code is making wrong edits, verbose mode reveals why. Maybe it read the wrong file, or its grep pattern matched something unexpected. Seeing the raw tool calls lets you diagnose and redirect: “You’re editing the wrong file — the auth middleware is in `src/middleware/auth.ts`, not `src/utils/auth.ts`.”

Most of the time, you don’t need verbose mode — Claude Code’s summaries are sufficient. But when you want to understand the machine, `Ctrl+O` pulls back the curtain.

Part 3: Advanced Techniques

Chapter 9: The Memory System

Every conversation with Claude Code starts fresh. You explain your architecture, share your preferences, clarify a naming convention — and next session, all of that is gone. Claude Code is brilliant within a conversation, but between conversations, it has amnesia.

The Memory system fixes this. It gives Claude Code persistent knowledge that survives across sessions — your preferences, your project's quirks, decisions you've made, context that would be tedious to re-explain every time. Combined with CLAUDE.md (which you learned in Chapter 5), Memory transforms Claude Code from a stateless tool into something that genuinely learns your project over time.

The Problem Memory Solves

Consider a typical week of development:

- Monday: You tell Claude Code "we use Prisma, not Drizzle" when it suggests the wrong ORM.
- Tuesday: You explain that the `payments` service talks to Stripe through a custom wrapper, not directly.
- Wednesday: You clarify that integration tests should use a real database, not mocks — because mocked tests passed but production broke.
- Thursday: You mention that the team is in a merge freeze until April 15.

Without Memory, you'd re-explain all of this in every new session. With Memory, Claude Code remembers. Monday's correction about Prisma persists into Tuesday. Wednesday's testing lesson carries forward indefinitely. Thursday's merge freeze stays active until you tell Claude Code the freeze is over.

Memory is the difference between working with a colleague who takes notes and one who doesn't.

How Memory Works

Memory files live at `~/.claude/projects/<project-hash>/memory/`. Each project gets its own memory directory, keyed by a hash of the project path. Inside that directory:

- `MEMORY.md` is the index file. The first 200 lines of this file are loaded into every conversation automatically. This is where Claude stores the most important context — a summary of what it knows about you and your project.

- **Individual topic files** store detailed knowledge about specific subjects. These are loaded on-demand when they become relevant to the conversation. If you're discussing testing, Claude loads the testing-

related memory file. If you're working on the API, the API-specific file gets loaded.

Memory is populated in two ways:

1. **Automatic** — Claude Code detects important information during conversations and saves it without being asked. If you correct a mistake, explain a preference, or share project context, Claude may save that for future reference.

2. **Explicit** — You tell Claude to remember something directly:

```
Remember that we use Prisma, not Drizzle, for all database access.
```

```
Remember: integration tests must use real databases, not mocks.
```

```
Remember that the team has a merge freeze until April 15.
```

Claude Code saves these as persistent memory entries that it will recall in future sessions.

Memory Types

Not all knowledge is the same. Claude Code organizes memory into four categories:

Type	Purpose	Example
`user`	Who you are, your expertise	"Senior backend dev, prefers Go over Python"
`feedback`	How Claude should approach work	"Don't mock the DB in tests — use test containers"
`project`	Ongoing project context	"Merge freeze starts April 15, unfreeze May 1"
`reference`	External resources and tools	"Bug tracking in Linear, project CORE-backend"

These types help Claude Code prioritize and apply knowledge correctly. A `feedback` memory about testing style is applied whenever Claude writes tests. A `project` memory about a merge freeze is applied whenever Claude tries to commit or push code.

Memory File Format

Each memory file uses a simple markdown format with YAML frontmatter:

```
---
name: testing-preferences
description: How to write tests in this project
type: feedback
---

Always use integration tests with real database, not mocks.

**Why:** Mocked tests passed but production migration failed — the
mock didn't enforce foreign key constraints.

**How to apply:** For database operations, use the test container
defined in `test/setup.ts`. For external APIs (Stripe, SendGrid),
use the VCR-style recorder in `test/fixtures/`.
```

The frontmatter tells Claude what the memory is about (`name`, `description`) and how to apply it (`type`). The body contains the actual knowledge, ideally with reasoning (`**Why:**`) and actionable guidance (`**How to apply:**`).

Another example — a reference memory:

```
---
name: deployment-process
description: How we deploy to production
type: reference
---

Deployments go through three stages:
1. PR merged to `main` triggers staging deploy (automatic)
2. QA team tests on staging (manual, ~2 hours)
3. Release manager runs `/deploy prod` in Slack

Never push directly to `main`. Never skip staging.
Contact @sarah-ops for emergency hotfix deployments.
```

The MEMORY.md Index

`MEMORY.md` is the master index. Claude Code reads the first 200 lines of this file at the start of every conversation, so it needs to be concise and high-signal. Think of it as a cheat sheet — one line per fact, organized by category.

```
# Project Memory

## User
- Senior backend developer, 8 years experience, prefers Go
- Uses Neovim in tmux, dark theme, minimal UI

## Project
- Merge freeze: April 15 - May 1 (spring release)
- New auth system migration in progress (see auth-migration)
- Performance budget: API responses under 200ms p99

## Feedback
- Use real DB in tests, not mocks (see testing-preferences)
- Prefer early returns over nested if/else
- Always add OpenAPI annotations to new endpoints

## Reference
- Bug tracking: Linear, project CORE-backend
- CI/CD: GitHub Actions, deploys via ArgoCD
- Monitoring: Datadog dashboards in #ops-alerts Slack channel
```

Keep each entry under 150 characters. The parenthetical references like `(see testing-preferences)` point to individual memory files that Claude loads on-demand when that topic comes up.

Stay under 200 lines total. Everything beyond line 200 is not loaded automatically — it would only be found if Claude explicitly reads the file. Put your most important context in the first 100 lines.

What NOT to Save in Memory

Memory is for knowledge that Claude Code can't get from other sources. Don't save:

- **Code patterns** — Claude can read your codebase directly. Instead of memorizing "we use the repository pattern," just let Claude see your repository classes.
- **Git history** — Claude has `git log`. Don't memorize "we merged the auth refactor last week." Claude can look that up.
- **Debugging solutions** — The fix is in the code. If you fixed a bug, the fix exists in the diff. Don't memorize "we fixed the null pointer in UserService" — it's already in git history.

- **Anything in CLAUDE.md** — CLAUDE.md is loaded every session, just like MEMORY.md. Don't duplicate information between them. CLAUDE.md is for static project rules. Memory is for dynamic, learned context.
- **Temporary information** — "The CI is broken today" doesn't need to be memorized. It'll be fixed tomorrow. Memory is for knowledge with at least a multi-week shelf life.

The rule of thumb: if Claude could discover it by reading code or running a command, don't put it in memory. Memory is for the things that exist only in your head.

Controlling Memory

You have several ways to manage what Claude Code remembers:

View and edit memory:

```
/memory
```

This opens a browser-like interface showing all memory files for the current project. You can read, edit, and delete entries.

Explicit save and forget:

```
Remember that we switched from REST to GraphQL for the mobile API.
```

```
Forget the merge freeze — it's been lifted.
```

Claude Code processes these as direct instructions to update or remove memory entries.

Disable auto-memory entirely:

If you don't want Claude Code saving information automatically, add this to your settings.json:

```
{
  "autoMemoryEnabled": false
}
```

With auto-memory disabled, Claude Code only saves memory when you explicitly ask it to. This gives you full control over what persists.

Memory and context compaction:

When a conversation gets long and Claude Code compacts the context (removing older messages to stay within the context window), memory survives. Key memories are re-injected from MEMORY.md after compaction, so Claude doesn't lose critical context even in marathon sessions.

Memory + CLAUDE.md: The Full Picture

These two systems serve different purposes and work together:

	CLAUDE.md	Memory
Content	Static project rules	Dynamic learned context
Updates	You edit manually	Claude updates automatically (or on request)
Scope	Shared with team (committed to git)	Personal (per-machine)
Loaded	Every session, full file	MEMORY.md first 200 lines + on-demand files
Examples	Build commands, coding conventions	Your preferences, project status, past decisions

Together, they give Claude Code two layers of knowledge: what the project needs (CLAUDE.md) and what you specifically need (Memory). A new team member cloning the repo gets the CLAUDE.md rules immediately. Your personal Memory — built over weeks of working together — stays on your machine, tuned to your workflow.

TIP:

Think of CLAUDE.md as the employee handbook and Memory as personal notes. The handbook tells everyone the rules. Your notes remind you of the decisions, preferences, and lessons learned along the way. Claude Code reads both before every conversation — which means it starts each session knowing the rules AND your personal context.

Chapter 10: Hooks — Automate Everything

Hooks are shell commands that run automatically at specific points in Claude Code's lifecycle. When Claude edits a file, a hook can auto-format it. When Claude runs a command, a hook can block it if it's dangerous. When Claude finishes working, a hook can run your test suite.

If CLAUDE.md teaches Claude Code what to do, hooks enforce how it's done. They're the mechanism that turns "please follow our code style" from a suggestion into an automated guarantee.

This is one of Claude Code's most powerful features, and one of its least known. Most users never touch hooks. The ones who do tend to wonder how they ever worked without them.

What Hooks Replace

Without hooks, you rely on Claude Code to remember your rules. "Always run prettier after editing." "Don't touch files in the deploy/ directory." "Run tests before committing." Claude Code usually follows these instructions — but "usually" isn't "always."

Hooks make it automatic. Format-on-edit. Block-on-violation. Test-on-completion. No relying on memory. No forgetting. Every edit, every command, every session — the hooks fire.

Hook Events

Hooks fire at specific lifecycle events. Here's when each one triggers:

Event	When It Fires	Typical Use Case
`PreToolUse`	Before a tool executes	Block dangerous commands, validate inputs
`PostToolUse`	After a tool succeeds	Auto-format, auto-lint, log actions
`Notification`	Claude sends a notification	Slack alert, desktop popup, sound
`Stop`	Claude finishes a turn	Auto-run tests, verify state
`SubagentStop`	A subagent finishes	Validate subagent output
`SessionStart`	Session begins	Inject context, check prerequisites
`UserPromptSubmit`	User sends a prompt	Pre-process input, add context

The most impactful events for daily use are `PostToolUse` (auto-formatting), `PreToolUse` (blocking dangerous operations), and `Stop` (auto-testing).

Hook Types

Hooks come in four varieties, each suited to different automation needs:

`command` — Run a shell command. This is the most common type. The command receives context via stdin (as JSON) and can influence Claude Code's behavior through its exit code and stderr output.

```
{
  "type": "command",
  "command": "prettier --write $CLAUDE_FILE_PATH"
}
```

`prompt` — Make a single LLM call. Useful when you need an AI decision without spawning a full conversation. The LLM evaluates the situation and returns a judgment.

```
{
  "type": "prompt",
  "prompt": "Review this edit for security issues. Return APPROVE or DENY with reason."
}
```

`agent` — Spawn a subagent. This is a full Claude Code instance that can read files, run commands, and make decisions. Use this for complex verification that requires multi-step reasoning.

```
{
  "type": "agent",
  "prompt": "Verify this change doesn't break any existing tests. Run the test suite and repor
        t."
}
```

`http` — POST to a webhook. Sends the hook context as JSON to a URL. Use this for external integrations — logging to a service, triggering CI, notifying a dashboard.

```
{
  "type": "http",
  "url": "https://hooks.slack.com/services/T00/B00/xxxx"
}
```

Configuration

Hooks are configured in your settings.json file. Each hook specifies which event it fires on, an optional matcher to filter by tool name, and the hook action:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "prettier --write $CLAUDE_FILE_PATH"
          }
        ]
      }
    ]
  }
}
```

This hook fires after every `Edit` tool call and runs Prettier on the edited file. The `$CLAUDE_FILE_PATH` variable is populated automatically with the path of the file that was just modified.

You can have multiple hooks on the same event:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "prettier --write $CLAUDE_FILE_PATH"
          },
          {
            "type": "command",
            "command": "eslint --fix $CLAUDE_FILE_PATH"
          }
        ]
      }
    ]
  }
}
```

Both Prettier and ESLint run after every edit, in order.

Five Practical Hook Recipes

Here are five hooks you can drop into your settings.json today.

1. AUTO-FORMAT ON EDIT

The most popular hook. Every file Claude edits gets formatted automatically — no “please run prettier” needed.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "npx prettier --write $CLAUDE_FILE_PATH 2>/dev/null || true"
          }
        ]
      }
    ]
```

```
    }
  }
```

The `2>/dev/null || true` ensures the hook doesn't fail on files Prettier doesn't support (images, binary files, etc.).

For Python projects, swap Prettier for Black:

```
{
  "type": "command",
  "command": "black $CLAUDE_FILE_PATH 2>/dev/null || true"
}
```

2. AUTO-LINT WITH ERROR FEEDBACK

This hook runs ESLint after edits and feeds any errors back to Claude Code, creating a self-correcting loop:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "npx eslint $CLAUDE_FILE_PATH --format compact 2>&1 || true"
          }
        ]
      }
    ]
  }
}
```

When ESLint finds issues, its output is fed back to Claude Code as hook output. Claude sees the lint errors and fixes them in the next edit. This creates an automatic format-lint-fix cycle without any manual intervention.

3. DESKTOP NOTIFICATION WHEN CLAUDE NEEDS INPUT

When Claude Code sends a notification (waiting for permission, asking a question), this hook triggers a macOS desktop notification:

```
{
  "hooks": {
    "Notification": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"Claude Code needs your attention\"
        with title \"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

On Linux, replace `osascript` with `notify-send`:

```
{
  "type": "command",
  "command": "notify-send 'Claude Code' 'Needs your attention'"
}
```

This is invaluable when you're working in another window while Claude Code runs a long task.

4. BLOCK EDITS TO PROTECTED FILES

Some files should never be modified by AI — production configs, lock files, security-critical code. This hook blocks edits to protected paths:

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "echo $CLAUDE_FILE_PATH | grep -qE '(\\.env|lock\\.json|yarn\\.lock|pack
        age-lock|deploy/prod)' && echo 'BLOCKED: Protected file' >&2 && exit 2 || exit 0"
          }
        ]
      }
    ]
```

```
    }
  }
```

Exit code 2 tells Claude Code to block the action. The message on stderr is shown to Claude as the reason for the block. Claude Code sees “BLOCKED: Protected file” and knows to find an alternative approach.

5. AUDIT LOG OF ALL TOOL CALLS

Track everything Claude Code does in a log file — useful for reviewing what happened in long autonomous sessions:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo \"$(date +%H:%M:%S) $CLAUDE_TOOL_NAME: $CLAUDE_FILE_PATH\" >> /tm
        p/claude-audit.log"
          }
        ]
      }
    ]
  }
}
```

After a session, check `/tmp/claude-audit.log` to see a chronological record of every tool Claude used and which files it touched.

Hook Input and Output

When a hook runs, it receives a JSON payload on stdin containing the full context of the event:

```
{
  "session_id": "abc123",
  "cwd": "/Users/dev/myproject",
  "hook_event_name": "PostToolUse",
  "tool_name": "Edit",
  "tool_input": {
    "file_path": "/Users/dev/myproject/src/api/users.ts",
    "old_string": "const users = db.query(...)",
    "new_string": "const users = await db.query(...)"
```

```
    }
  }
```

This means your hooks can make intelligent decisions based on context — which tool ran, what file was affected, what the edit contained.

Exit codes control behavior:

Exit Code	Meaning
`0`	Allow — the action proceeds normally
`2`	Block — the action is denied. Reason is read from stderr
Other	Error — logged but action still proceeds

Hook output is fed back to Claude. Whatever your hook prints to stdout becomes part of Claude Code's context. This is the mechanism that makes the lint feedback loop work — ESLint's output goes to stdout, Claude reads it, and Claude fixes the issues.

The Feedback Loop

The most powerful pattern with hooks is the automatic feedback loop. Here's how it works:

1. Claude edits a file
2. `PostToolUse` hook runs the linter
3. Linter finds 2 errors and prints them to stdout
4. Claude Code receives the linter output
5. Claude reads the errors and makes a second edit to fix them
6. `PostToolUse` hook runs the linter again
7. Linter passes — zero errors
8. Claude moves on

This happens automatically. You don't intervene. Claude edits, gets feedback, corrects, and the linter confirms. Every edit converges toward clean code.

The same pattern works for type checking:

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          {
            "type": "command",
            "command": "cd $CLAUDE_PROJECT_DIR && npx tsc --noEmit --pretty 2>&1 | head -20 ||
        true"
          }
        ]
      }
    ]
  }
}
```

Now every edit is type-checked. If Claude introduces a type error, it sees the TypeScript compiler output and fixes it immediately. You get type-safe edits without ever asking.

Hooks Override Permissions

Here's a critical insight that most users miss: **hooks operate above the permission system.** Even in the most permissive mode where Claude Code has full autonomy, a `PreToolUse` hook that returns exit code 2 will still block the action.

This means hooks are your ultimate safety net. You can give Claude Code broad permissions for speed while using hooks to enforce hard boundaries:

- Full autonomy for edits and commands — but hooks block modifications to `deploy/`, `.env`, and `*.lock`
- Full autonomy for git operations — but a hook prevents force-pushes to `main`
- Full autonomy for file creation — but a hook prevents creating files larger than 100KB

Permissions say what Claude Code is allowed to do. Hooks enforce what it actually does. When they conflict, hooks win.

TIP:

Start with one hook — auto-format on edit. It's low-risk, high-reward, and teaches you how hooks work. Once you're comfortable, add lint feedback. Then protected file blocking. Build your hook collection incrementally rather than configuring everything at once.

Chapter 11: MCP Servers — Extend Claude Code Infinitely

Out of the box, Claude Code can read files, write files, search code, and run shell commands. That's powerful for pure code work. But real-world development involves more than code — you interact with databases, issue trackers, deployment platforms, documentation wikis, messaging systems, and dozens of other tools.

MCP — the Model Context Protocol — lets Claude Code talk to all of them. It's an open standard that gives AI models a way to connect with external tools and data sources through a simple, unified interface. Think of MCP servers as USB ports for Claude Code: plug in a PostgreSQL server, and Claude can query your database. Plug in a GitHub server, and Claude can search issues and create pull requests. Plug in a Slack server, and Claude can read and send messages.

The result is a Claude Code that doesn't just work with your code — it works with your entire development environment.

Why MCP Matters

Without MCP, when you need Claude Code to check something in your database, the workflow looks like this:

1. You switch to your database client
2. Run a query
3. Copy the results
4. Paste them into Claude Code
5. Claude analyzes and responds

With MCP, Claude Code queries the database directly:

```
What are the most common error types in the logs table from the last 24 hours?
```

Claude Code runs the SQL query through the MCP server, gets the results, and analyzes them — all in one step. No context switching, no copy-pasting, no manual query writing.

This same pattern applies to every external tool. Instead of being the manual bridge between Claude Code and your infrastructure, MCP lets Claude Code reach out directly.

Configuration

MCP servers are configured in your settings.json file under the `mcpServers` key. Each server has a name, a command to start it, and optional environment variables:

```
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "postgresql://user:pass@localhost:5432/mydb"
      }
    }
  }
}
```

When Claude Code starts, it launches each configured MCP server as a subprocess. The servers communicate with Claude Code using JSON-RPC over stdin/stdout — a simple, fast protocol that works everywhere.

You can configure multiple servers simultaneously:

```
{
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "postgresql://user:pass@localhost:5432/mydb"
      }
    },
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "ghp_xxxxxxxxxxxx"
      }
    },
    "slack": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-slack"],
      "env": {
        "SLACK_BOT_TOKEN": "xoxb-xxxxxxxxxxxx"
      }
    }
```

```
    }
  }
```

Now Claude Code can query your database, manage GitHub issues, and send Slack messages — all from the same conversation.

Popular MCP Servers

The MCP ecosystem is growing rapidly. Here are the servers you're most likely to use:

Server	Package	What It Does
PostgreSQL	`@modelcontextprotocol/server-postgres`	Run SQL queries, inspect schema, manage data
GitHub	`@modelcontextprotocol/server-github`	Issues, PRs, code search, repository management
Slack	`@modelcontextprotocol/server-slack`	Read channels, send messages, search history
Filesystem	`@modelcontextprotocol/server-filesystem`	Sandboxed file access with configurable root
Puppeteer	`@modelcontextprotocol/server-puppeteer`	Browser automation, screenshots, web scraping
SQLite	`@modelcontextprotocol/server-sqlite`	Query SQLite databases, inspect tables
Memory	`@modelcontextprotocol/server-memory`	Persistent key-value store for cross-session data
Vercel	`@vercel/mcp`	Deployments, logs, environment variables, domains

Community-built servers extend this further — there are MCP servers for Notion, Jira, Linear, MongoDB, Redis, Docker, Kubernetes, and dozens more. The registry at `mcp.so` lists available servers.

Practical Examples

Database exploration:

```
Show me the schema of the users table and the 10 most recent
signups with their subscription status.
```

With the PostgreSQL MCP server configured, Claude Code runs `\d users` to get the schema, then executes `SELECT * FROM users ORDER BY created_at DESC LIMIT 10`, joined with the subscriptions table. You get a formatted table without writing any SQL yourself.

Issue triage:

```
Find all open GitHub issues labeled "bug" that were created in
the last week. Summarize them and suggest which ones I should
prioritize.
```

With the GitHub MCP server, Claude Code fetches the issues via the GitHub API, reads their descriptions and comments, and provides a prioritized summary.

Deployment monitoring:

```
Check the latest Vercel deployment. Did it succeed? Show me any
error logs if it failed.
```

With the Vercel MCP server, Claude Code checks deployment status and pulls logs — saving you from navigating the Vercel dashboard.

Building Your Own MCP Server

If an existing MCP server doesn't cover your needs, building one is straightforward. MCP servers expose tools via JSON-RPC — each tool has a name, a description, and input/output schemas.

Here's a minimal MCP server in TypeScript that exposes a single tool:

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from
  "@modelcontextprotocol/sdk/server/stdio.js";
```

```typescript
import { z } from "zod";

const server = new McpServer({
  name: "weather",
  version: "1.0.0"
});

server.tool(
  "get_weather",
  "Get current weather for a city",
  { city: z.string().describe("City name") },
  async ({ city }) => {
    const res = await fetch(
      `https://wttr.in/${city}?format=j1`
    );
    const data = await res.json();
    const current = data.current_condition[0];
    return {
      content: [{
        type: "text",
        text: `${city}: ${current.temp_C}°C, ` +
              `${current.weatherDesc[0].value}`
      }]
    };
  }
);

const transport = new StdioServerTransport();
await server.connect(transport);
```

In Python, the equivalent uses the `mcp` package:

```python
from mcp.server.fastmcp import FastMCP
import httpx

mcp = FastMCP("weather")

@mcp.tool()
async def get_weather(city: str) -> str:
    """Get current weather for a city."""
    async with httpx.AsyncClient() as client:
        r = await client.get(
            f"https://wttr.in/{city}?format=j1"
        )
        data = r.json()
        c = data["current_condition"][0]
        return f"{city}: {c['temp_C']}°C, " \
               f"{c['weatherDesc'][0]['value']}"
```

```
mcp.run()
```

Configure your custom server the same way:

```
{
  "mcpServers": {
    "weather": {
      "command": "node",
      "args": ["./my-mcp-servers/weather.js"]
    }
  }
}
```

Now Claude Code can check the weather. More practically, you can build MCP servers that connect to your company's internal APIs, proprietary databases, or custom tooling that no public server supports.

Security Considerations

MCP servers have the same access as any process you run. A PostgreSQL MCP server can read and write your database. A filesystem MCP server can access files within its configured root. A GitHub MCP server can create issues and PRs with your token.

Review what you connect. Don't blindly install MCP servers from unknown sources. Each server gets real credentials and real access to your systems.

Use environment variables for credentials. Never hardcode tokens or passwords in settings.json:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    }
  }
}
```

Use `settings.local.json` for personal secrets. Project-level settings.json is committed to git — anyone who clones the repo sees it. Put your personal API keys and tokens in `settings.local.json`, which should be gitignored:

```
.claude/settings.local.json  ← your personal tokens (gitignored)
.claude/settings.json         ← shared team config (committed)
```

Scope access narrowly. If a GitHub MCP server only needs to read issues, create a token with `issues:read` scope — not full repository access. Principle of least privilege applies to AI tools just like it does to human access.

Managing MCP Servers

Use the `/mcp` slash command to see all configured MCP servers, their status, and the tools they expose:

```
/mcp
```

This shows which servers are running, which tools are available, and any connection errors.

You can also specify an MCP configuration file when launching Claude Code:

```
claude --mcp-config ./mcp.json
```

This is useful for project-specific MCP setups, CI environments, or when switching between different server configurations.

To restart a specific MCP server without restarting Claude Code:

```
/mcp restart postgres
```

Useful when you've updated the server configuration or the server process crashed.

> **TIP:**
> Start with one MCP server that solves a real friction point in your workflow. If you're constantly copy-pasting database query results into Claude Code, add the PostgreSQL server. If you're manually checking GitHub issues, add the GitHub server. One well-configured MCP server saves more time than five you barely use.

Chapter 12: IDE Integration

Claude Code isn't locked to one interface. You can use it from a raw terminal, inside VS Code, within JetBrains IDEs, as a native desktop app, or through the web at claude.ai. Each interface has different strengths, and understanding them lets you pick the right one for the situation — or combine multiple for the best experience.

Available Platforms

Platform	Best For	Key Advantage
CLI (terminal)	Power users, scripting	Full feature access, fastest startup, composable with pipes
Desktop app	Daily development	Native system tray, multiple windows, drag-and-drop files
VS Code	IDE-first developers	Inline code actions, sidebar panel, deep editor integration
JetBrains	IntelliJ/PyCharm users	Tool window, code selection context, all JetBrains IDEs
Web (claude.ai/code)	Remote access, quick tasks	No install, cloud execution, scheduled tasks

No single interface is universally "best." Power users tend to gravitate toward the CLI for its speed and scriptability, but many developers prefer the visual integration of VS Code or JetBrains. The web interface is ideal when you're not at your development machine. The desktop app sits in between — more visual than the CLI, more flexible than an IDE extension.

VS Code Integration

The VS Code extension puts Claude Code in a sidebar panel within your editor. Install it from the VS Code marketplace by searching for "Claude Code" or from the command line:

```
code --install-extension anthropic.claude-code
```

Sidebar panel: Claude Code appears as a panel in your VS Code sidebar. You can chat with it, see file edits in real-time, and approve or reject changes — all without leaving the editor. The conversation history persists across panel toggles.

Inline code actions: Right-click any code selection and you'll see Claude Code actions in the context menu. "Explain this code," "Refactor this function," "Add tests for this selection" — these send the selected code directly to Claude Code with your instruction.

```
Right-click on a function → Claude Code → "Add error handling"
```

Claude Code receives the selected code plus its file path and surrounding context, making its edits more precise than if you described the code verbally.

Terminal integration: VS Code's integrated terminal runs Claude Code natively. You can have the sidebar panel open for visual feedback while running CLI commands in the terminal — two interfaces to the same underlying engine.

File context: When you have files open in VS Code, Claude Code can see which tabs are active. This helps it understand what you're currently working on. If you have `users.ts` and `users.test.ts` open, Claude Code infers you're working on the users module.

JetBrains Integration

Claude Code works in all JetBrains IDEs — IntelliJ IDEA, PyCharm, WebStorm, GoLand, RubyMine, PhpStorm, CLion, and Rider. The integration is a plugin available from the JetBrains Marketplace.

Tool window: Claude Code appears as a tool window (similar to the terminal or git panels). You can dock it to any side of the IDE, resize it, or pop it out into a separate window.

Code selection context: Select code in the editor and send it to Claude Code with a keyboard shortcut. The selection, file path, and surrounding context are included automatically — just like the VS Code inline actions.

Project-aware: The plugin reads your project's structure, detecting the framework, language, build system, and dependencies. This information is available to Claude Code without you describing it.

Configuration: JetBrains plugin settings let you configure the model, permission mode, and other options through the IDE's preferences panel — no need to edit settings.json directly.

Desktop Application

The Claude Code desktop app is a native application that runs independently of any IDE. It provides a dedicated, always-available interface to Claude Code.

System tray: The desktop app lives in your system tray (macOS menu bar or Windows taskbar). Click the icon to open a conversation instantly. This makes Claude Code feel like a utility that's always one click away, not a tool you need to launch.

Multiple sessions: Open multiple windows for different projects or conversations. Work on a frontend task in one window and a backend task in another — each with its own context, history, and project scope.

Drag and drop: Drop files directly into the desktop app window. Screenshots, PDFs, images, code files — the app reads them and makes them available to Claude Code. This is the fastest way to share visual context like error screenshots or design mockups.

Launch from terminal: The `/desktop` command in the CLI opens the desktop app with your current project context:

```
claude
> /desktop
```

This bridges the CLI and desktop workflows — start in the terminal, switch to the desktop app when you want a richer visual experience.

Web Interface

Claude Code is available on the web at claude.ai, with a code-specific mode that connects to a cloud execution environment. This is the only interface that doesn't require a local installation.

Remote cloud execution: Code runs on Vercel's cloud infrastructure, not your local machine. This means you can use Claude Code from a tablet, a Chromebook, or any device with a web browser — no development environment needed.

Scheduled tasks: The web interface supports the `/schedule` command, which creates tasks that run on a cron schedule without your machine being online:

```
/schedule "Run the test suite and report failures" every day at 9am
```

This creates a cloud-based Claude Code agent that executes the task on schedule and sends you the results. Useful for daily status checks, automated reporting, and maintenance tasks.

No local CLI needed: For quick questions, code reviews, or tasks that don't need access to your local filesystem, the web interface is the fastest path. Open a browser, paste some code, get an answer.

Limitations: The web interface runs in a sandboxed cloud environment. It doesn't have access to your local files, local databases, or local services. For tasks that require your actual development environment, use the CLI or an IDE extension.

Why Experts Prefer the CLI

Every interface is built on top of the CLI. The VS Code extension, the JetBrains plugin, and the desktop app all use the same Claude Code engine underneath. But the CLI exposes capabilities that the graphical interfaces don't.

Fastest startup. No extension loading, no GUI rendering. `claude` in the terminal and you're working in under a second.

Works over SSH. When you're debugging on a remote server, the CLI works in your SSH session. No port forwarding, no remote desktop, no workarounds. SSH in, run `claude`, and you have full Claude Code access on the remote machine.

Full feature access. Every Claude Code flag, every configuration option, every edge case is accessible from the CLI. IDE extensions surface the most common features but necessarily simplify the interface. The CLI hides nothing.

Scriptable with pipes. The CLI composes with standard Unix tools. This opens up workflows that graphical interfaces can't match:

```
# One-shot query — no interactive session
echo "Explain this error" | claude -p

# Pipe code for review
git diff | claude -p "Review this diff for bugs"

# Pipe a file for analysis
cat logs/error.log | claude -p "What caused these errors?"

# Generate and use output in scripts
claude --print "Write a regex that matches email addresses" \
  > /tmp/regex.txt
```

The `-p` flag (or `--print`) runs Claude Code in non-interactive mode — it reads stdin, processes the prompt, prints the result to stdout, and exits. No conversation, no session, just input-output. This makes Claude Code a composable Unix tool.

Combine with other commands:

```
# Find all TODO comments and ask for prioritization
grep -rn "TODO" src/ | claude -p "Prioritize these TODOs"

# Analyze test coverage gaps
npm test -- --coverage 2>&1 | claude -p "What areas need more tests?"

# Review a specific commit
git show abc1234 | claude -p "Review this commit"
```

Multiple CLI flags not available in IDE extensions:

```
# Specify model
claude --model claude-sonnet-4-20250514

# Set permission mode
claude --allowedTools "Edit,Read,Grep,Glob"

# Resume previous conversation
claude --resume

# Continue most recent conversation
claude --continue

# Use specific MCP config
claude --mcp-config ./custom-mcp.json

# Output format for scripting
claude -p "query" --output-format json
```

These flags give you precise control over every aspect of Claude Code's behavior — control that graphical interfaces abstract away.

Multi-Environment Setup

Most developers use Claude Code across multiple machines and projects. Here's how configuration travels:

Global CLAUDE.md (`~/.claude/CLAUDE.md`) — Your personal preferences. This stays on your machine and applies to every project. It doesn't sync across machines automatically, but you can symlink it to a dotfiles repository:

```
ln -s ~/dotfiles/claude/CLAUDE.md ~/.claude/CLAUDE.md
```

Now your Claude Code preferences sync wherever you sync your dotfiles.

Project CLAUDE.md (repo root) — Travels with the repository. When you clone a project on a new machine, `git clone` brings the CLAUDE.md with it. Every team member gets the same instructions.

Memory (`~/.claude/projects/`) — Per-machine, per-project. Memory does not sync between machines. If you use both a laptop and a desktop, each builds its own memory independently. This is by design — memory contains machine-specific context that might not apply elsewhere.

Settings (`.claude/settings.json`) — Can be committed to git for team-wide settings. Use `.claude/settings.local.json` for personal overrides that shouldn't be shared (API keys, personal tool preferences).

A typical team setup:

```
Committed to git:
  CLAUDE.md                    ← project rules (team-shared)
  .claude/settings.json        ← shared settings (hooks, MCP)
  .claude/rules/*.md           ← path-specific rules

Not committed (gitignored):
  .claude/settings.local.json  ← personal API keys, overrides
  ~/.claude/CLAUDE.md          ← global personal preferences
  ~/.claude/projects/*/memory/ ← auto-memory (per-machine)
```

This separation means the team shares rules and configurations, while individual preferences and secrets stay personal.

Choosing Your Setup

Here's a decision framework:

You're a terminal power user — Use the CLI as your primary interface. Keep a terminal pane dedicated to Claude Code in your tmux or terminal multiplexer. Use pipes for one-shot tasks.

You want visual integration — Use VS Code or JetBrains. The inline code actions and sidebar panel make Claude Code feel like part of the IDE. Fall back to the CLI for advanced flags and scripting.

You want always-available access — Use the desktop app. System tray presence means Claude Code is one click away regardless of what you're doing.

You work on multiple machines — Use the web interface for quick tasks from any device. Use the CLI or IDE extension on your primary development machines.

Best of all worlds — Use the IDE extension for daily development, the CLI for scripting and automation, and the desktop app or web interface for quick access. They all share the same CLAUDE.md and project configuration, so switching between them is seamless.

> **TIP:**
>
> Even if you use an IDE daily, learn the CLI. It's the most powerful interface, with features that IDE extensions don't expose. The pipe workflow alone — `git diff | claude -p "review this"` — is worth the learning curve. You'll find yourself reaching for it whenever you need something fast, scriptable, or precise.

Part 4: Expert Secrets

Chapter 13: Prompt Engineering for Claude Code

The difference between a developer who gets mediocre results from Claude Code and one who gets exceptional results almost always comes down to prompting. Not because Claude Code requires special syntax or magic words, but because how you frame a request determines how much context Claude Code can leverage — and context is everything.

This chapter covers the prompting patterns that expert developers use daily. These aren't theoretical guidelines. They're battle-tested techniques drawn from thousands of real development sessions.

How Claude Code Prompts Differ

If you've used ChatGPT, Claude.ai, or any web-based AI tool, you've learned a certain prompting style: describe what you want in detail, provide background information, maybe include code snippets. That style works against you in Claude Code.

Claude Code is not a chatbot. It has tools. It can read your files, search your codebase, run commands, check git history, and inspect directory structures. It has persistent state — it remembers what you discussed earlier in the session. It knows which project you're in, what language you're using, and what framework your code is built on.

This means you don't need to provide context that Claude Code can discover on its own. Instead, you need to point it in the right direction and tell it what to accomplish.

Bad prompt:

```
Write a function that validates email addresses. It should check for
an @ symbol, a valid domain, and return true or false. Use TypeScript.
```

This is how you'd prompt a web chatbot. You're describing the implementation because the chatbot can't see your code. Claude Code can see your code.

Good prompt:

```
Add email validation to the signup handler in src/auth/signup.ts.
Use the same validation pattern as src/auth/utils.ts.
```

This prompt is shorter but far more effective. Claude Code will read both files, understand your existing validation patterns, and produce code that fits seamlessly into your codebase. The result will match your code style, your error handling patterns, your type conventions — all without you describing any of them.

The shift is fundamental: **stop describing the solution. Describe the goal and point to the context.**

The 5 Principles of Expert Prompting

After working with hundreds of developers' Claude Code sessions, clear patterns emerge in what separates excellent prompts from average ones. These five principles account for the vast majority of quality difference.

PRINCIPLE 1: BE SPECIFIC ABOUT LOCATION

Claude Code can search your entire codebase, but telling it exactly where to work eliminates ambiguity and saves time. Include file paths, function names, and even line numbers when you know them.

Vague: "Fix the login bug."

Specific: "Fix the race condition in `src/auth/login.ts` in the `handleSubmit` function — the loading state isn't reset when the API call fails."

The specific prompt gets Claude Code working immediately. The vague prompt forces it to search for what "login bug" might mean, potentially finding the wrong issue or asking you clarifying questions.

This principle applies to feature requests too:

Vague: "Add a dark mode toggle."

Specific: "Add a dark mode toggle to the settings panel in `src/components/Settings.tsx`. Store the preference in the existing `usePreferences` hook from `src/hooks/usePreferences.ts`."

The more precisely you specify location, the fewer decisions Claude Code has to guess about — and the fewer guesses mean fewer mistakes.

PRINCIPLE 2: REFERENCE EXISTING PATTERNS

Your codebase already has patterns — how you write API routes, how you structure components, how you handle errors. Telling Claude Code to follow existing patterns produces code that reads like a human on your team wrote it.

```
Add a PATCH /users/:id endpoint to src/api/users.ts.
```

```
Follow the same pattern as the PATCH /posts/:id endpoint in src/api/posts.ts.
```

Claude Code reads the referenced file, extracts the pattern (middleware chain, validation approach, response format, error handling), and applies it to the new endpoint. The result is consistent with your codebase rather than consistent with some generic best practice that doesn't match your style.

This works at every scale:

- **Components:** "Create a ProductCard component following the pattern of `UserCard` in `src/components/UserCard.tsx`."
- **Tests:** "Add tests for the payment module. Follow the test structure in `tests/auth.test.ts`."
- **Migrations:** "Add a `last_login` column to the users table. Follow the migration format in `migrations/003_add_email_verified.sql`."

When you don't reference a pattern, Claude Code uses its general knowledge of best practices. When you do reference a pattern, it uses your team's specific conventions. The latter is almost always better.

PRINCIPLE 3: STATE THE WHY

Including the reason behind your request helps Claude Code make better implementation decisions — especially when there are multiple valid approaches.

Without why: "Add caching to the user profile endpoint."

With why: "Add caching to the user profile endpoint — we're hitting the database 50 times per second for the same profiles during peak traffic. Use a 5-minute TTL since profiles rarely change."

The "why" tells Claude Code that this is a performance optimization, that the cache should be keyed per user, that staleness of up to 5 minutes is acceptable, and that the primary concern is database load rather than response latency. All of that context is implicit in two sentences.

Here's another example:

Without why: "Extract this function into a separate module."

With why: "Extract the PDF generation logic from `src/api/reports.ts` into `src/services/pdf.ts` — three other routes need PDF generation and they're all duplicating this logic."

Now Claude Code knows to design the extracted module with a clean public API that works for multiple callers, not just the one it was extracted from.

PRINCIPLE 4: SCOPE YOUR REQUEST

The right-sized request is specific enough to be clear but broad enough to be useful. Too narrow and you end up micromanaging. Too broad and Claude Code has to make too many decisions without guidance.

Too narrow: “On line 47 of `src/api/users.ts`, change `findOne` to `findUnique`.”

You could do this yourself faster than typing the prompt. Only use this level of granularity when you need Claude Code to understand the change in context of a larger task.

Too broad: “Refactor the entire backend to use the repository pattern.”

This is a multi-day project with hundreds of decisions. Claude Code will either ask many clarifying questions or make assumptions you disagree with. Break it up.

Right-sized: “Refactor `src/api/users.ts` to use a repository pattern. Extract the database queries into `src/repositories/UserRepository.ts`. Keep the same API contract — no changes to request/response shapes.”

This is one coherent unit of work with clear boundaries. Claude Code can complete it in a single session without guessing about scope.

A good rule of thumb: if the task would take a human developer 15 minutes to 2 hours, it’s probably right-sized for a single Claude Code prompt. Smaller tasks aren’t worth the overhead. Larger tasks should be broken into sequential prompts.

PRINCIPLE 5: GIVE SUCCESS CRITERIA

Tell Claude Code what “done” looks like. This is especially important for features with multiple edge cases or tasks where correctness matters more than speed.

```
Add a payment processing function to src/services/payment.ts.

Test cases to cover:
- Successful payment (charge amount, return confirmation)
- Card declined (return specific error code)
- Invalid amount (negative, zero, exceeds maximum)
- Duplicate transaction ID (idempotency check)

Run npm test after to verify everything passes.
```

The success criteria serve three purposes. First, they tell Claude Code what edge cases to handle in the implementation. Second, they define what tests to write. Third, the “run npm test” instruction makes Claude Code verify its own work before telling you it’s done.

Without success criteria, Claude Code will handle the happy path and maybe one or two obvious error cases. With them, it builds a complete, production-quality implementation.

Anti-Patterns to Avoid

Certain prompting habits actively reduce the quality of Claude Code's output. Here are the most common ones.

Don't start with "Can you..." — Claude Code doesn't need politeness. "Can you add error handling to the login function?" becomes "Add error handling to the login function in `src/auth/login.ts`." The direct version is clearer and gets better results. You're not being rude — you're being precise.

Don't explain how Claude Code works to Claude Code. Prompts like "You have access to my filesystem, so please read the file first and then..." are wasted tokens. Claude Code knows what tools it has. Just state the goal.

Don't paste errors without context. Pasting a stack trace with "fix this" sometimes works, but adding one sentence of context makes it work reliably: "Getting this error when running `npm test` after adding the new user endpoint" followed by the stack trace. The context tells Claude Code where to look and what changed.

Don't ask Claude Code to "be careful" or "double-check." These phrases don't make Claude Code more careful. If you want verification, ask for specific verification: "Run the test suite after making changes" or "Verify the migration is reversible." Actionable instructions beat vague caution.

Don't bundle unrelated tasks. "Fix the login bug, add a dark mode toggle, and update the README" combines three independent tasks. Handle them in separate prompts. Each task gets full attention, and if one goes wrong, it doesn't contaminate the others.

Don't over-specify implementation details. "Use a for loop to iterate over the array and check each element" is micromanaging. "Filter users by active status" lets Claude Code choose the idiomatic approach for your language and codebase.

Power Prompt Templates

These four templates cover the vast majority of development tasks. Adapt them to your needs.

BUG FIX TEMPLATE

```
Fix [symptom] in [file/function].
Expected: [correct behavior]
```

```
Actual: [what happens instead]
Reproduce: [command or steps]
```

Example:

```
Fix the 500 error on POST /api/orders in src/api/orders.ts.
Expected: Returns 201 with order confirmation.
Actual: Crashes with "Cannot read property 'id' of undefined."
Reproduce: npm test -- --grep "create order"
```

FEATURE TEMPLATE

```
Add [feature] to [location].
Follow the pattern in [reference file].
Include tests covering: [cases].
Run [verification command] when done.
```

Example:

```
Add a PATCH /users/:id/avatar endpoint to src/api/users.ts.
Follow the upload pattern in src/api/posts.ts (multer + S3).
Include tests for: valid image, file too large, invalid format, unauthorized.
Run npm test when done.
```

REFACTOR TEMPLATE

```
Extract [what] from [source] into [destination].
Reason: [why this refactor matters].
Keep [constraints — what shouldn't change].
```

Example:

```
Extract email sending logic from src/api/auth.ts and src/api/orders.ts
into src/services/email.ts.
```

```
Reason: both files duplicate SMTP setup and template rendering.
Keep the same function signatures — callers shouldn't need changes.
```

DEBUG TEMPLATE

```
Debugging [error/symptom] when [action/trigger].
Context: [what changed recently or what you've already checked].
Find the root cause and fix it.
```

Example:

```
Debugging intermittent timeout on the /api/search endpoint.
Context: started after adding pagination in commit abc123.
Only happens when page > 10. Already checked — DB query returns fast.
Find the root cause and fix it.
```

Conversation Management

How you manage your conversation with Claude Code is as important as individual prompts. Long conversations accumulate context, which is powerful — but they also accumulate noise.

Use `/compact` when context gets long. After 20-30 exchanges, your conversation carries a lot of history. The `/compact` command summarizes the conversation into key points and frees up context space. Use it when you notice Claude Code starting to lose track of earlier decisions or when responses slow down.

Start new conversations for new topics. If you’ve been debugging a database issue and now want to add a UI feature, start a fresh conversation. The database debugging context will only confuse the UI work. Use `claude` to start fresh, or `/clear` to reset in the same session.

Use `@filename` to bring files into context. When you mention a file path naturally, Claude Code reads it. But `@src/api/users.ts` explicitly loads the file into the conversation at that point, ensuring Claude Code has the latest version front and center.

Use `/cost` to monitor token usage. If you’re on a metered plan, `/cost` shows how much the current session has consumed. Useful for budgeting long sessions.

Keep prompts conversational within a session. Your first prompt in a session should be detailed (include file paths, context, success criteria). Follow-up prompts can be terse because Claude Code remembers

the context: “Now add the same validation to the update endpoint” or “Also handle the case where the user doesn’t exist.”

> **TIP:**
> The single most impactful change you can make to your prompting is adding file paths. “Fix the auth bug” might work. “Fix the auth bug in `src/auth/middleware.ts`” almost always works. When in doubt, include the path.

Chapter 14: Parallel Development with Subagents & Worktrees

Most development work is sequential. You work on a feature, finish it, start the next one. Even with Claude Code, the default mode is one conversation, one task at a time. But many tasks have independent parts that could happen simultaneously — writing tests for module A while implementing module B, reviewing code in one area while refactoring another, researching an approach while building a prototype.

Claude Code’s subagent system and git worktree integration unlock true parallel development. This chapter shows you how to run multiple Claude Code instances simultaneously, each working on independent tasks, without them interfering with each other.

The Problem with Sequential Work

Consider a typical feature implementation:

1. Write the API endpoint (20 minutes)
2. Write tests for the API endpoint (15 minutes)
3. Update the frontend component (15 minutes)
4. Write tests for the frontend component (10 minutes)
5. Update the documentation (5 minutes)

Sequential, that’s 65 minutes. But steps 3-5 don’t depend on steps 1-2. The frontend work needs the API contract, not the implementation. The docs need the feature description, not the finished code. In theory, you could do steps 1-2 and 3-5 in parallel — cutting the wall-clock time significantly.

Without tooling, parallelism is hard. You’d need multiple terminal windows, separate git branches, manual coordination. It’s more effort than it’s worth for most tasks. Subagents and worktrees make it effortless.

Agent Tool — Spawning Subagents

Claude Code can spawn specialized sub-agents: independent Claude Code instances that work on isolated tasks without polluting your main conversation's context.

When you ask Claude Code to handle something that benefits from isolation — a research question, a parallel task, a code review — it can delegate to a subagent. The subagent gets its own fresh context, works independently, and reports back when finished.

You can trigger subagents explicitly:

```
Run tests for the auth module in the background while I work on the payment feature.
```

Claude Code spawns a subagent to run the tests. Your main conversation continues uninterrupted. When the tests finish, you're notified with the results.

Or implicitly — Claude Code sometimes recognizes that a task has independent sub-tasks and spawns agents on its own, especially in plan execution mode.

Each subagent is fully isolated. It has its own conversation history, its own tool context, and its own working state. Changes one subagent makes to files are visible to others (they share the filesystem), but their conversation contexts don't leak. This means a subagent researching your authentication architecture doesn't fill your main conversation with hundreds of lines of code exploration.

Agent Types

Not all subagents are the same. Different agent types are optimized for different kinds of work:

Type	Purpose	Best For
`general-purpose`	Multi-step tasks requiring tools	Code changes, implementations, complex research
`Explore`	Codebase exploration and understanding	“How does the auth system work?”, architecture questions
`Plan`	Architecture and design planning	“Design the approach for adding multi-tenancy”
`code-reviewer`	Reviewing completed code changes	Pre-commit review, PR review, security audit

Explore agents are lightweight. They read files and search code but don’t make changes. Use them when you need to understand something about your codebase without modifying it. They’re fast because they don’t need write permissions.

Plan agents produce structured plans — step-by-step implementation approaches with file lists, dependency analysis, and risk assessment. They don’t write code. They plan what code to write. Use them before starting complex features.

Code reviewer agents analyze changes with a critical eye. They look for bugs, performance issues, security vulnerabilities, and style inconsistencies. Spawn one after completing a feature to get a review before you commit.

General-purpose agents can do everything — read, write, run commands, make decisions. They’re the default when a task requires actual code changes.

Background Agents

The most practical form of parallel development is background agents. You launch a task in the background and keep working in the foreground. When the background task finishes, you’re notified.

Launching from conversation:

```
Run the full test suite in the background and report any failures.
I'll keep working on the migration.
```

Claude Code spawns a background agent that runs the tests. Your conversation continues. When tests finish, you see the results inline.

Using Ctrl+B:

If you're in the middle of a task and realize it's going to take a while, press `Ctrl+B` to send it to the background. Claude Code continues working on that task in a background process, and your terminal is freed up to start a new conversation. You're notified when the background task completes.

This is surprisingly useful in practice. You ask Claude Code to refactor a large module, it starts working, and you realize you don't need to watch it. `Ctrl+B` — now you can start a new `claude` session and work on something else while the refactor continues.

Checking status:

Background agents run independently. When they finish, Claude Code shows their results. If you're in the middle of typing when a background agent completes, the notification waits until you're at a natural break point.

Git Worktrees — Isolated Environments

Background agents share your filesystem. If agent A is editing `src/api/users.ts` while agent B is also editing `src/api/users.ts`, you have a conflict. Git worktrees solve this.

A git worktree is an isolated copy of your repository in a separate directory, on a separate branch, but sharing the same git history. Think of it as a lightweight clone that doesn't duplicate the entire `.git` directory.

```
claude --worktree feature-auth
```

This creates a new worktree in a sibling directory (e.g., `../myproject-feature-auth/`), checks out a new branch `feature-auth`, and starts Claude Code in that isolated directory. Everything you do in that session happens in the worktree — edits, commits, test runs — all isolated from your main working directory.

When you're done, the changes exist on the `feature-auth` branch. Merge them back to main via git merge or a pull request, just like any other branch.

Why worktrees instead of separate clones? Worktrees share the git object store. Creating one is instant — no network fetch, no disk duplication. They're also aware of each other, so git prevents you from checking out the same branch in multiple worktrees (which would cause confusion).

The `.worktreeinclude` file: Your main directory might have gitignored files that the worktree needs — `.env` files, local configs, build caches. Create a `.worktreeinclude` file in your project root listing the gitignored files that should be copied into new worktrees:

```
.env
.env.local
node_modules/
```

When Claude Code creates a worktree, it copies these files over so the worktree environment matches your main one.

The `/batch` Command — Parallel Changes at Scale

Sometimes you need the same type of change across many files or modules. Migrating a component library, updating API call patterns, converting class components to functional ones. The `/batch` command automates this at scale.

```
/batch migrate all Vue components in src/components/ to React
```

Here's what happens: Claude Code analyzes the scope (finds all Vue components), creates a plan (one worktree per component or group of components), spawns subagents (each in its own worktree, each handling a subset of the migration), and opens pull requests (one per worktree, each independently reviewable).

The power of `/batch` is that each subagent works in complete isolation. If the migration of `UserProfile.vue` fails, it doesn't affect the migration of `Dashboard.vue`. Each PR can be reviewed, tested, and merged independently.

`/batch` can handle 5 to 30 parallel worktrees depending on your machine's resources. For large-scale changes — framework migrations, API version upgrades, dependency replacements — this turns days of work into hours.

Other practical `/batch` use cases:

```
/batch add error handling to all API routes in src/api/
/batch add unit tests for all services in src/services/
```

```
/batch update all imports from lodash to lodash-es
/batch add TypeScript types to all .js files in src/utils/
```

Each of these spawns parallel agents, each in its own worktree, each producing an independent PR.

When to Use What

The parallel development toolkit has several options. Here's a decision guide:

Quick answer or exploration — Use an Explore agent in the foreground. "How does the caching layer work?" doesn't need background processing. Get the answer, move on.

Independent work that takes time — Use a general-purpose agent in the background. "Run the full test suite," "Refactor the logging module," "Add comprehensive tests to the payment service." Send it to the background with `Ctrl+B` and keep working.

Feature work that might conflict with your current work — Use a worktree. If you're editing `src/api/users.ts` and want Claude Code to also refactor `src/api/users.ts` in a different way, you need isolation. Worktrees give you separate branches and separate directories.

Mass changes across many files — Use `/batch`. When the same type of change applies to 10+ files or modules, `/batch` parallelizes the work and produces independent PRs.

Pre-commit review — Use a code-reviewer agent. After finishing a feature, spawn a reviewer to check your work before committing. It runs in the foreground (you want to read the review), but it doesn't pollute your main conversation's context.

Complex feature planning — Use a Plan agent before starting implementation. Have it analyze the codebase, identify dependencies, and produce a step-by-step plan. Then execute the plan yourself or hand it to an executing agent.

Custom Subagents

You can define custom agent types with specialized instructions, tool restrictions, and behavioral guidelines. Create a markdown file in `.claude/agents/`:

```
.claude/agents/security-reviewer/AGENT.md
```

```
---
name: security-reviewer
```

```
description: Reviews code changes for security vulnerabilities
model: claude-sonnet-4-6
tools: Read Grep Glob
---

## Review Checklist

When reviewing code, systematically check for:

1. **SQL Injection** — Parameterized queries? No string concatenation in SQL?
2. **XSS** — Output encoding? React's JSX auto-escaping intact?
3. **CSRF** — Token validation on state-changing endpoints?
4. **Auth bypass** — Every endpoint checks permissions?
5. **Secrets** — No hardcoded API keys, passwords, or tokens?
6. **Input validation** — All user input validated and sanitized?
7. **Rate limiting** — Abuse-prone endpoints rate-limited?

Report findings as:
- CRITICAL: Must fix before merge
- WARNING: Should fix, but not blocking
- INFO: Suggestion for improvement
```

Now you can invoke this agent:

```
Use the security-reviewer agent to review my changes before I commit.
```

Claude Code spawns a subagent with the `security-reviewer` configuration. It only has read access (Read, Grep, Glob — no Edit, no Bash), so it can't accidentally modify your code while reviewing it. It follows your specific review checklist rather than generic review heuristics.

Custom agents are particularly powerful for teams. Define agents for your team's specific concerns — performance review, accessibility audit, API contract verification — and every team member gets consistent, specialized review.

The `model` field lets you choose the right model for the agent's task. Use a faster, cheaper model for simple checks and a more capable model for complex analysis. The `tools` field restricts what the agent can do — a reviewer that can only read is safer than one that can also edit.

TIP:

Start with background agents before exploring worktrees. The simplest parallel workflow is "run tests in the background while I keep coding." Once that feels natural, add worktrees for feature

isolation. `/batch` is the advanced tool — reach for it when you have 10+ similar changes to make. Build your parallelism skills incrementally.

Chapter 15: Custom Skills & Superpowers

Every developer has workflows they repeat. Deploy to production. Review a pull request. Set up a new microservice. Debug a performance issue. These workflows have steps, checks, and decisions that you've refined over time — but they live in your head or in scattered documentation.

Skills turn these workflows into reusable recipes that Claude Code follows precisely. Instead of remembering "run tests, then build, then deploy, then verify the health endpoint, then check the logs," you invoke a skill and Claude Code executes the entire workflow. Skills are one of Claude Code's most underused features, and for teams, they're transformative.

What Skills Are

A skill is a markdown file with structured instructions that Claude Code loads and follows when invoked. Think of skills as "runbooks for AI" — step-by-step procedures that Claude Code executes with full access to your codebase and tools.

Skills differ from CLAUDE.md instructions in two important ways. First, skills are loaded on demand — they don't consume context until you invoke them. Second, skills have metadata that controls when and how they run, including tool permissions, model selection, and activation conditions.

You invoke skills with a slash command:

```
/deploy
```

Or by describing a task that matches a skill's description — Claude Code recognizes the match and loads the appropriate skill automatically.

Built-in Skills

Claude Code ships with a library of built-in skills that cover common development workflows. You don't need to create these — they're available immediately.

`/brainstorming` — Structured creative exploration. Before building a feature, this skill guides you through requirements gathering, constraint identification, and approach evaluation. It prevents the common mistake of jumping into code before understanding what you're building.

```
/brainstorming — I need to add multi-tenancy to our SaaS app
```

The skill asks targeted questions: What isolation level do you need? Shared database or separate? How do tenants authenticate? What data is tenant-specific vs global? By the end, you have a clear specification rather than a vague idea.

`/writing-plans` — Detailed implementation plans. Takes a feature specification and produces a step-by-step plan with file changes, dependency analysis, risk assessment, and testing strategy. The plan is concrete enough to hand to an executing agent or follow manually.

`/executing-plans` — Step-by-step plan execution with review checkpoints. Reads a plan (from a file or conversation) and executes it incrementally, pausing at checkpoints for your review. Useful for complex changes where you want to verify each stage before proceeding.

`/test-driven-development` — Red-green-refactor cycle. Writes failing tests first, then implements the code to make them pass, then refactors. This skill enforces TDD discipline even when you're tempted to skip straight to implementation.

`/systematic-debugging` — Methodical investigation framework. Instead of random hypothesis testing, this skill follows a structured debugging process: reproduce, isolate, identify root cause, verify fix, check for similar issues. It prevents the common trap of fixing symptoms instead of causes.

`/verification-before-completion` — Verify before claiming done. This skill runs after implementation to confirm that everything actually works — tests pass, the feature behaves correctly, edge cases are handled. It catches the "it compiles so it must work" false confidence.

`/simplify` — Reviews recent changes for unnecessary complexity, code duplication, and optimization opportunities. A post-implementation cleanup pass that catches over-engineering.

`/batch` — Parallel changes across multiple worktrees, as covered in Chapter 14.

These built-in skills chain naturally. A typical workflow for a new feature:

```
/brainstorming → /writing-plans → /executing-plans → /verification-before-completion
```

Each skill builds on the output of the previous one. Brainstorming produces requirements. Writing-plans produces a plan from those requirements. Executing-plans implements the plan. Verification confirms it works.

How Skills Work Under the Hood

A skill is a markdown file with optional YAML frontmatter. When invoked, Claude Code loads the file's content as instructions and follows them for the duration of the task. The instructions can include steps, decision trees, checklists, and code templates.

Here's the structure:

```
---
name: my-skill
description: What this skill does (Claude reads this to match tasks)
allowed-tools: Bash(npm *) Bash(git *) Edit Read
---

## Instructions

The actual workflow steps go here.
Claude Code follows these as its guide.
```

The frontmatter controls metadata. The markdown body contains the workflow. Claude Code reads both and executes the workflow using the permitted tools.

Skills are loaded only when invoked — they don't consume context during normal conversation. This means you can have dozens of skills defined without any performance penalty. Only the active skill's instructions occupy context space.

Creating Your Own Skills

Custom skills live in your home directory or your project:

- **Personal skills:** `~/.claude/skills/skill-name/SKILL.md`
- **Project skills:** `.claude/skills/skill-name/SKILL.md`

Personal skills are available in every project. Project skills are available only in that project and can be committed to git for the whole team.

Here's a practical example — a deployment skill:

```
---
name: deploy
description: Deploy the application to production
allowed-tools: Bash(npm *) Bash(git *) Bash(curl *) Read
---

## Pre-deployment Checks

1. Verify all tests pass:
   ```bash
   npm test
```
```
If any test fails, STOP and report the failure. Do not proceed.

2. Verify the build succeeds:

```bash
npm run build
```

3. Check for uncommitted changes:

```bash
git status --porcelain
```

If there are uncommitted changes, STOP and ask the user to commit first.

4. Verify you're on the main branch:

```bash
git branch --show-current
```

If not on main, STOP and warn the user.

Deploy

5. Deploy:

```bash
npm run deploy
```

Post-deployment Verification

6. Wait 30 seconds for deployment to propagate.

7. Check the health endpoint:

   ```
   curl -sf https://api.example.com/health
   ```

   If the health check fails, immediately report and suggest rollback.

8. Check the version endpoint:

   ```
   curl -sf https://api.example.com/version
   ```

   Verify the deployed version matches the current git commit.

Report

Summarize: what was deployed, which commit, health check status, any warnings.

```
Now `/deploy` executes this entire workflow. Every deployment follows the same process — tests,
build, branch check, deploy, health verification. No steps forgotten, no checks skipped.

### Skill Features in Depth

The frontmatter supports several powerful features:

| Feature | Purpose | Example |
|---------|---------|---------|
| `description` | Tells Claude when to use the skill | `"Deploy to production with safety check
s"` |
| `allowed-tools` | Pre-approve tools (no permission prompts) | `Bash(npm *) Edit Read Grep` |
| `model` | Use a specific model for this skill | `claude-sonnet-4-6` |
| `effort` | Set reasoning effort level | `high` |
| `context: fork` | Run in an isolated subagent | Skill doesn't pollute main context |
| `paths` | Only activate for matching file paths | `["src/api/**", "src/services/**"]` |
| `disable-model-invocation` | Only the user can invoke, not Claude | Prevents accidental invoc
ation |

**`allowed-tools`** is particularly important. When a skill specifies allowed tools, those tool
s don't require user approval while the skill is active. Your deployment skill can run `npm tes
```

t`, `npm run build`, and `npm run deploy` without pausing for permission at each step. This mak
es skills feel smooth and automatic rather than interrupted by approval dialogs.

Tool patterns support globs: `Bash(npm *)` allows any npm command. `Bash(git *)` allows any git
command. `Bash(curl https://api.example.com/*)` allows curl but only to your API domain. This g
ives you fine-grained control over what the skill can do.

`context: fork` runs the skill in a subagent. The skill's execution doesn't fill your main
conversation with tool calls and intermediate output. You see the final result, not the journe
y. Use this for skills that involve many steps and produce a lot of intermediate output.

`paths` restricts when the skill activates. A skill with `paths: ["*.test.ts", "*.spec.ts"]
` only triggers when you're working on test files. This prevents skills from activating in the
wrong context.

`disable-model-invocation` prevents Claude Code from invoking the skill on its own. Without
this flag, Claude Code might automatically use a skill when it recognizes a matching task. With
the flag, only explicit `/skill-name` invocations trigger it. Use this for destructive skills
(deployment, database migrations) where you want human intent.

Dynamic Context in Skills

Skills can include live data from your environment using shell execution syntax. When Claude Co
de loads the skill, it runs the embedded commands and injects their output into the instruction
s.

```markdown
---
name: review-pr
description: Review the current pull request
allowed-tools: Read Grep Glob Bash(gh *)
---

## PR Context

- **Diff:** !`gh pr diff`
- **PR description:** !`gh pr view`
- **Comments:** !`gh pr view --comments`
- **CI status:** !`gh pr checks`

## Review Instructions

Review the diff above for:
1. Logic errors and edge cases
2. Security vulnerabilities
3. Performance concerns
4. Test coverage gaps
5. Code style consistency
```

```
Provide feedback organized by severity: critical, warning, suggestion.
```

The `!` backtick syntax executes the command when the skill loads. By the time Claude Code starts following the instructions, it has the full PR diff, description, comments, and CI status in context. The review is grounded in actual PR data, not hypothetical code.

This makes skills context-aware. A deployment skill can check the current branch and environment. A review skill can load the actual diff. A debugging skill can capture recent error logs. The skill's instructions adapt to the current state of your project.

String Substitutions

Skills support variable substitutions for dynamic behavior:

- **`$ARGUMENTS`** — Everything the user typed after the skill name. `/deploy staging` makes `$ARGUMENTS` equal to `"staging"`.
- **`$0`** — The full argument string (alias for `$ARGUMENTS`).
- **`$1`**, **`$2`**, ... — Individual arguments, space-separated. `/deploy staging --force` makes `$1` = `"staging"` and `$2` = `"--force"`.
- **`${CLAUDE_SESSION_ID}`** — Unique ID for the current session. Useful for creating unique filenames or log entries.
- **`${CLAUDE_SKILL_DIR}`** — The directory containing the skill file. Useful for referencing helper scripts or templates that ship alongside the skill.

Here's a skill that uses arguments:

```
---
name: new-service
description: Scaffold a new microservice
allowed-tools: Bash(*) Edit Write Read
---

## Create Service: $1

1. Create directory structure:
```

src/services/$1/ ├── index.ts ├── routes.ts ├── service.ts ├── types.ts └── **tests/** └── service.test.ts ```

2. Use the template files in ${CLAUDE_SKILL_DIR}/templates/ as starting points.

3. Register the service in src/services/index.ts.

4. Add a health check route at GET /$1/health.

```
Invoking `/new-service payments` scaffolds a complete payments microservice. The skill knows wh
ere to put it, what files to create, and how to register it — all from the instructions.

### When Skills Shine

Skills deliver the most value in four scenarios:

**Repeatable workflows.** Deployment, release management, database migration, environment setu
p. Any workflow you do more than twice should be a skill. The first time you write the skill ta
kes 10 minutes. Every subsequent execution saves 15-30 minutes and eliminates human error.

**Team standards.** When your team has specific processes — PR review checklists, code style re
quirements, architecture patterns — skills encode them into executable instructions. New team m
embers don't need to learn the process from documentation. They invoke the skill and Claude Cod
e guides them through it.

**Complex processes with many steps.** Some workflows have 10+ steps, conditional branches, and
verification checks. Humans forget steps. Humans skip checks. Skills don't. A deployment skill
that checks 8 things before deploying will check 8 things every time.

**Onboarding.** New developers on a team can invoke skills to understand how things work. `/set
up-dev-environment` configures everything. `/architecture-overview` explains the codebase. `/ad
d-feature` scaffolds a new feature following team conventions. Skills accelerate onboarding fro
m weeks to days.

### Building a Skill Library

Start with one skill for your most painful repeated workflow. Deploy it, use it for a week, ref
ine it based on experience. Then add a second skill for your second-most painful workflow. Buil
d your library incrementally rather than trying to skill-ify everything at once.

A typical mature skill library:
```

~/.claude/skills/ ├── deploy/SKILL.md ← Production deployment ├── new-component/SKILL.md ← Scaffold React component ├── new-api-route/SKILL.md ← Scaffold API endpoint ├── review-security/SKILL.md ← Security-focused review └── debug-performance/SKILL.md ← Performance investigation

.claude/skills/ (project-specific, committed to git) ├── setup-dev/SKILL.md ← Dev environment setup ├── run-migrations/SKILL.md ← Database migration workflow └── release/SKILL.md ← Release process

```
Personal skills in `~/.claude/skills/` travel with you across projects. Project skills in `.cla
ude/skills/` travel with the repository and are shared with the team. The combination gives you
personal productivity and team consistency.

> **Tip:** When writing a skill, be explicit about failure conditions. Don't just say "run the
tests." Say "run the tests. If any test fails, STOP and report the failure. Do not proceed to t
he next step." Claude Code follows instructions literally — if you don't specify what to do on
failure, it might continue past a broken step. Every step in a skill should have a clear succes
s condition and a clear failure action.

---


## Chapter 16: 50 Hidden Tips & Tricks Most Developers Don't Know

This is the chapter you came for. These aren't in the getting-started guide. They're collected
from expert usage, documentation deep-dives, and real-world workflows. If you already know more
than ten of these, you're in the top 1% of Claude Code users.

### Speed & Efficiency

**1. The `!` prefix runs shell commands inline.**
Type `!` followed by any shell command and the output lands directly in your conversation conte
xt. Claude sees it immediately and can act on it. Faster than switching terminals and pasting o
utput.
```

! git log –oneline -5 ! ls src/components/ ! npm test – –grep “auth”

```


**2. `/fast` uses the same model — it's not a downgrade.**
Many think fast mode switches to a weaker model. It doesn't — same Claude, optimized for faster
output with slightly less thoroughness. Use it for straightforward tasks like renaming variable
s or adding imports. Switch back for architecture decisions.
```

/fast rename the variable `data` to `userProfile` across all files in src/

```


**3. `Shift+Tab` cycles models instantly.**
```

```
Switch between Opus, Sonnet, and Haiku mid-conversation without restarting. Use Haiku for quick
questions, Sonnet for standard tasks, Opus for complex architecture. Haiku costs roughly 1/60th
of Opus — strategic switching cuts your bill dramatically.
```

Shift+Tab → cycles: Opus → Sonnet → Haiku → Opus

```
**4. `/compact` accepts a focus argument.**
Most developers use `/compact` bare and let Claude decide what to keep. Add a focus argument to
control what matters most during compression. Without it, Claude might discard exactly the cont
ext you need next.
```

/compact focus on the authentication refactor and the database schema changes /compact keep all the API endpoint signatures and test results

```
**5. Pipe mode for one-shot queries.**
The `-p` flag pipes stdin directly into Claude and prints the result to stdout. No interactive
session needed. This is transformative for integrating Claude into shell workflows — it compose
s with any Unix tool.

```bash
git diff | claude -p "review this diff for bugs"
cat error.log | claude -p "what's causing these failures?"
git log --oneline -20 | claude -p "categorize these commits" | pbcopy
```
```

6. `claude --print` for non-interactive scripting. Pass a prompt directly and get the result on stdout. No interactive session, no stdin needed. The `-o json` flag forces JSON output for script consumption.

```
claude --print "what node version does this project use?"
claude --print -o json "list all TODO comments in src/" > todos.json
```

7. Resume any conversation with `--resume`. Every conversation is automatically saved. Resume with full context — files read, decisions made, plans discussed — exactly where you left off. You can close your terminal, reboot your machine, or switch projects and pick up right where you were. Name important sessions so you can find them later.

```
claude --resume                    # resume last conversation
claude -r                          # shorthand
claude --resume "auth-refactor"    # resume a named session
```

8. `@filename` forces a file into context. The `@` prefix explicitly loads a file into context. Use it when Claude's suggestions don't match your conventions — force it to see the reference file. You can reference multiple files in one prompt.

```
@src/auth/middleware.ts why doesn't the new route handler follow this pattern?
@src/types.ts @src/schema.ts add a new User type matching the database schema
```

9. Edit is 10x cheaper than Write. Edit sends only the diff. Write sends the entire file. For a 500-line file where you're changing 3 lines, Edit transmits ~10 lines while Write transmits all 500. Reinforce this in your CLAUDE.md:

```
# Rules
- Always use Edit for modifying existing files
- Only use Write for creating new files
```

10. Parallel tool calls happen automatically. When Claude needs to read 5 files, it reads all 5 simultaneously — not sequentially. You don't configure this; it's built into the architecture. The same applies to searches: multiple grep operations fire in parallel. Give Claude the full list and it analyzes everything together in one round-trip.

```
Check these files for inconsistent error handling:
src/api/users.ts, src/api/orders.ts, src/api/payments.ts,
src/api/inventory.ts, src/api/notifications.ts
```

Configuration Secrets

11. Three layers of settings merge top-down. Claude Code has three config files that merge in a specific order. Understanding this hierarchy is essential for teams. Project settings override global and are shared via git. Local settings override everything and stay private.

```
~/.claude/settings.json          # Global (your machine)
.claude/settings.json            # Project (commit this to git!)
```

```
.claude/settings.local.json      # Local (gitignored, personal overrides)
```

12. Allowlist specific commands with regex. `permissions.allow` accepts patterns, not just exact strings. Auto-approve safe command families while keeping destructive ones gated.

```
{
  "permissions": {
    "allow": [
      "Read", "Glob", "Grep",
      "Bash(npm test *)",
      "Bash(git (add|commit|status|diff|log) *)",
      "Bash(cargo (build|test|clippy) *)"
    ]
  }
}
```

The git pattern allows common operations but still requires approval for `git reset --hard` or `git push --force`.

13. `.claudeignore` hides files from Claude. Create a `.claudeignore` file in your project root with the same syntax as `.gitignore`. Claude won't read, search, or reference matching files. This reduces noise, protects sensitive data, and keeps context focused. A 50,000-file monorepo with proper `.claudeignore` behaves like a focused 2,000-file project.

```
# .claudeignore
node_modules/
dist/
build/
*.lock
*.min.js
coverage/
.env*
vendor/
```

14. Custom keybindings. Remap any shortcut in `~/.claude/keybindings.json`. Supports modifiers and VS Code-style chord bindings (press one combo, release, press another).

```
[
  { "key": "ctrl+l", "command": "clear" },
  { "key": "ctrl+k ctrl+c", "command": "compact" },
  { "key": "ctrl+shift+p", "command": "plan" },
```

```
    { "key": "ctrl+d", "command": "diff" }
  ]
```

15. Auto-format every file Claude edits. One PostToolUse hook and every file Claude modifies gets auto-formatted. Replace `prettier` with `black`, `gofmt`, or `rustfmt` as needed.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "prettier --write $CLAUDE_FILE_PATH"
          }
        ]
      }
    ]
  }
}
```

16. Desktop notifications when Claude needs attention. Set up a Notification hook and get alerted when Claude pauses for input. On Linux, replace `osascript` with `notify-send`.

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "",
        "hooks": [
          {
            "type": "command",
            "command": "osascript -e 'display notification \"$CLAUDE_NOTIFICATION\" with title
        \"Claude Code\"'"
          }
        ]
      }
    ]
  }
}
```

17. Project-scoped MCP servers vs global ones. MCP servers in `.claude/settings.json` are project-scoped — they load only when you're in that project and are shared with your team via git. MCP in

`~/.claude/settings.json` loads everywhere. Connection strings and secrets go in `settings.local.json` (gitignored). This gives your team shared tooling with private credentials.

```
// .claude/settings.json (committed, shared with team)
{
  "mcpServers": {
    "team-database": {
      "command": "npx",
      "args": ["@team/db-mcp-server"],
      "env": { "DB_URL": "${DB_URL}" }
    }
  }
}
```

18. `claude config set` from the terminal. Configure without editing JSON. Respects the three-layer hierarchy — without `--global`, writes to project config; with it, writes to `~/.claude/settings.json`.

```
claude config set permissions.allow '["Read","Glob","Grep"]'
claude config set --global theme "dark"
claude config list
```

19. Multiple CLAUDE.md files merge automatically. Claude Code loads and merges instructions from multiple locations. Global for personal preferences, project for team conventions, subdirectory for module-specific rules.

```
~/.claude/CLAUDE.md              # Global (all projects)
./CLAUDE.md                      # Project root (team conventions)
./src/CLAUDE.md                  # Subdirectory (module-specific rules)
```

20. Path-specific rules with frontmatter. Create markdown files in `.claude/rules/` with `paths:` frontmatter. These rules only activate when Claude works on matching files. Non-matching rules are invisible — they consume zero context. This scales beautifully for large projects with varied conventions across modules.

```
---
paths: ["src/api/**/*.ts"]
---
All API handlers must validate input using Zod schemas.
Return standardized error responses using ApiError class.
```

```
---
paths: ["**/*.test.ts"]
---
Use describe/it blocks. Mock external services, never hit real APIs.
```

Advanced Workflows

21. `/init` generates your CLAUDE.md automatically. Scans your project structure, package.json, git history, and coding patterns — then generates a comprehensive CLAUDE.md file. The result includes detected language conventions, testing frameworks, build commands, and common patterns. Usually 60-70% correct out of the box. Always review and customize.

```
/init
```

22. Plan mode prevents wrong paths. For complex tasks, ask Claude to plan before coding. It produces a structured breakdown — files to change, order of operations, risks, and testing strategy — without modifying anything. Much cheaper to revise a plan than to undo a half-completed refactor.

```
Plan a refactor of the authentication system to support OAuth2.
Don't change any code yet — just outline the approach.
```

23. Background agents with `Ctrl+B`. Send a task to a background agent that works independently while you continue your main conversation. Perfect for mechanical tasks while you focus on creative work.

```
Main conversation: "Building the payment integration..."
Ctrl+B: "Update all copyright headers to 2026"
# Notification appears when done
```

24. `/branch` forks your conversation. Try two approaches without losing progress. Creates a named checkpoint you can return to if the current path doesn't work.

```
/branch explore-option-A
# ... try approach A, doesn't work? ...
/resume explore-option-A
# ... try approach B from the same starting point
```

25. `/rewind` jumps to any checkpoint. Not just undo — jump to any previous point in the conversation. Think of it as `git checkout` for conversations. Context from that point forward is discarded; you restart from a known good state.

```
/rewind
# Shows a list of checkpoints — select where to return
```

26. Claude reads images natively. Pass a path to any PNG/JPG and Claude sees it visually. This unlocks workflows that would otherwise require manual description: “recreate this UI,” “what’s wrong with this layout,” “match this design.” Claude analyzes visual composition — colors, spacing, typography — and generates matching code. Not pixel-perfect, but gets you 80% in one shot.

```
@screenshots/target-design.png recreate this layout in React with Tailwind CSS
```

27. Claude reads PDFs with page ranges. Work directly from specs and documentation in PDF format. Max 20 pages per request — for longer docs, read in chunks.

```
Read pages 12-18 of docs/api-specification.pdf and implement
the user endpoints described there.
```

28. Side questions with minimal context pollution. Need a quick answer mid-task without derailing focus? Frame it as a side question. Keeps your main conversation thread clean.

```
Quick aside: what's the syntax for TypeScript conditional types?
```

29. `/context` visualizes your context window. Shows a breakdown of what’s consuming context — CLAUDE.md, history, file contents, tool outputs — with capacity warnings. When approaching 60%, use `/compact` proactively (tip #4).

```
/context
```

30. `/diff` for visual change review. See all session modifications in visual diff format. Your pre-commit review — catch anything unexpected before Claude commits.

```
/diff
# Shows a colored diff of all changes in the session
```

Hidden Power

31. The hook feedback loop. PostToolUse hooks create automatic feedback loops: Claude edits, hook runs linter, linter reports error, Claude sees it, fixes it, hook runs again, passes. Fully automatic, typically resolves in 1-2 iterations.

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "eslint --fix $CLAUDE_FILE_PATH && eslint $CLAUDE_FILE_PATH"
          }
        ]
      }
    ]
  }
}
```

32. Hooks override ALL permission modes. Hooks returning exit code 2 block operations even with `--dangerously-skip-permissions`. Hooks sit above the permission system, not below it — the ultimate safety net. This hook prevents writing to any `.env` file, period. No flag can bypass it.

```
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Write",
        "hooks": [
          {
            "type": "command",
            "command": "if echo $CLAUDE_FILE_PATH | grep -q '.env'; then exit 2; fi"
          }
        ]
      }
    ]
```

```
  }
}
```

33. Claude never amends commits unless explicitly asked. When a pre-commit hook fails, the commit didn't happen. If Claude used `--amend` on retry, it would modify the previous (unrelated) commit. So Claude always creates a new commit. Intentional and non-configurable — prevents data loss.

```
# After a failed pre-commit hook:
git add fixed-file.ts
git commit -m "fix: resolve lint errors"   # NEW commit, not --amend
```

34. Context compression triggers at ~60%, not 95%. Automatic compression is aggressive — starts around 60% utilization, not near capacity. If you wait for it, Claude decides what to keep. Proactive compression is a strategy; automatic compression is a safety net.

```
/compact focus on: the API redesign decisions, the new schema,
and the migration strategy we discussed
```

35. Sub-agents for codebase exploration. Instead of manually searching through files, describe what you want to understand. Claude launches a focused investigation — searching, reading, following imports, checking test files for behavior clues, and synthesizing across multiple files. Far more effective than manual grep-and-read cycles.

```
Figure out how the payment processing pipeline works end-to-end.
Trace from the API endpoint through to the database write.
```

36. `--max-budget-usd` sets a hard spending limit. Cap session spending. Once reached, the session ends gracefully. Set low for exploration, higher for known-complex tasks. Essential for teams managing shared budgets.

```
claude --max-budget-usd 2.00
claude --max-budget-usd 0.50   # tight budget for quick tasks
```

37. `--json-schema` forces structured output. Constrains output to match your JSON schema exactly. No markdown, no extra text — just valid, parseable JSON. Combine with `--print` for automated pipelines.

```
claude -p --json-schema '{
  "type": "object",
  "properties": {
    "bugs": { "type": "array", "items": { "type": "string" } },
    "severity": { "type": "string", "enum": ["low","medium","high"] }
  }
}' "analyze src/auth.ts for bugs"
```

38. `/schedule` runs tasks on Anthropic's cloud. Schedule Claude on a cron — even when your computer is off. Runs on Anthropic infrastructure with access to your connected repositories. Tasks can create issues, leave PR comments, or post summaries depending on what you configure.

```
/schedule "review all open PRs and comment" every weekday at 9am
/schedule "check for outdated dependencies" every monday at 8am
```

39. Extended cloud planning with deep reasoning. Cloud-based planning gives Claude up to 30 minutes of reasoning time. For decisions that affect months of development — service boundaries, database architecture, major migrations.

```
/ultraplan migrate our monolith to microservices —
analyze all 200 endpoints and propose service boundaries
```

40. `/batch` for mass parallel changes. Spawns multiple parallel agents in separate git worktrees. Each handles a subset of files; results are presented as individual reviewable changes. For a 50-file migration, hours become minutes.

```
/batch update all test files from Jest to Vitest syntax
/batch add input validation to every API endpoint
```

Pro Patterns

41. "Follow the pattern in X" is the most powerful prompt. Reference an existing file and Claude replicates its structure, naming conventions, error handling, and style perfectly. Claude reads the reference file, extracts the pattern — imports, middleware chain, validation approach, response format — and applies it to the new code. Produces more consistent output than any amount of detailed description.

```
Create a new API endpoint for /api/orders.
Follow the exact pattern in src/api/users.ts.
```

42. Start conversations scoped to a single concern. Don't let one conversation handle auth, database optimization, and UI styling. Short, focused conversations produce better results than marathon multi-topic sessions. When in doubt, start fresh.

```
# Session 1: Auth refactor
claude → "refactor the authentication middleware..."
# Session 2: Database optimization (new conversation)
claude → "optimize the slow queries in reports..."
```

43. `!git diff` as context is better than explanation. Instead of describing what changed, show Claude the actual diff. More accurate and complete than verbal explanation.

```
!git diff HEAD~3
Review these changes. Are there any bugs or missed edge cases?

!git diff main..feature-branch
Write a comprehensive PR description for these changes.
```

44. When you deny a tool call, explain why. A bare denial forces Claude to guess what went wrong. An explained denial teaches it your preference immediately and adjusts behavior for the rest of the conversation. It's the fastest way to steer Claude's approach.

```
# Weak:
[Deny]

# Strong:
[Deny] — Don't use Write. Use Edit instead — I want the minimal diff.
[Deny] — That file is auto-generated. Modify src/schema.prisma instead.
```

45. Token cost scales with file size — use offset and limit. Reading a 5,000-line file consumes real tokens. If you only need 50 lines, say so. Dramatically cheaper and keeps context cleaner.

```
Read lines 200-250 of src/services/payment.ts — the refund handling logic.
```

46. `claude doctor` diagnoses configuration problems. Checks authentication, config files, MCP servers, and dependencies. Catches expired tokens, misconfigured settings, and incompatible versions. Run it first, debug second.

```
claude doctor
```

47. `.worktreeinclude` copies gitignored files to worktrees. When `/batch` spawns agents in worktrees, gitignored files like `.env` don't copy — because they're gitignored. Add them to `.worktreeinclude` at your project root. Without this, parallel agents fail because they can't find environment variables or generated files.

```
# .worktreeinclude
.env
.env.local
src/generated/prisma-client/
```

48. Track your usage with `/stats`. See session counts, token consumption, costs, and streaks. Helps you understand usage patterns and discover which task types cost more than expected.

```
/stats
```

49. Pre-commit hooks create an automatic quality gate. Claude respects git hooks. If a hook fails — linting, type checking, test execution — Claude sees the error output, fixes the issue, stages the fix, and retries the commit. All without your intervention. Strict hooks don't slow Claude down; they make its output more reliable.

```
# .husky/pre-commit
npm run lint
npm run typecheck
npm test -- --bail
```

50. Combine everything into a personalized AI development environment. The real power of Claude Code isn't any single feature — it's the combination. CLAUDE.md defines your conventions. Memory preserves

knowledge across sessions. Hooks enforce standards automatically. MCP servers connect external tools. Skills encode your workflows. Path-specific rules adapt behavior per module. A fully configured setup doesn't feel like a tool; it feels like a team member who knows your codebase and improves with every session. Chapter 20 shows the complete blueprint for building this from scratch.

```
CLAUDE.md          → What Claude should know
Memory             → What Claude should remember
Hooks              → What Claude must always do
MCP Servers        → What Claude can access
Skills             → How Claude should work
Settings layers    → How your team configures it all
```

Part 5: Real-World Mastery

Chapter 17: Building a Feature Start-to-Finish

Everything in this book so far has been modular — individual techniques, commands, and configurations you can learn in isolation. This chapter puts them all together. We're going to walk through building a real feature with Claude Code, from blank slate to merged pull request, showing the exact conversation flow at each stage.

The feature: add user notification preferences to a Next.js application. Users should be able to toggle email, push, and SMS notifications independently for three categories — marketing, product updates, and security alerts. The backend needs API routes, database schema changes, input validation, and tests.

This is a realistic mid-complexity feature. Not a trivial CRUD endpoint, not a massive architecture overhaul. The kind of thing a developer builds several times a week.

Phase 1: Planning

You open your terminal in the project directory and start Claude Code.

```
cd ~/projects/acme-app
claude
```

The first prompt sets the direction:

```
Plan notification preferences for our app. Users should toggle
email/push/SMS per category (marketing, updates, security).
I need: database schema, API routes, validation.
Look at our existing code structure before proposing anything.
```

Notice what this prompt does. It describes the feature clearly but doesn't dictate implementation. The instruction to "look at existing code structure" is critical — it tells Claude Code to investigate before proposing.

Claude Code reads your `package.json`, scans the `src/` directory, checks `prisma/schema.prisma` for your existing data model, and reads a few existing API routes to understand your conventions. It comes back with a plan:

```
Based on your codebase:
- Database: Prisma with PostgreSQL
- API routes: Next.js App Router in src/app/api/
- Validation: Zod schemas in src/lib/validations/
- Auth: NextAuth with session middleware

Proposed plan:
1. Add NotificationPreference model to Prisma schema
2. Create migration
3. Add Zod validation schema
4. Create GET /api/notifications/preferences
5. Create PUT /api/notifications/preferences
6. Add tests following patterns in tests/api/

Data model:
- userId (relation to User)
- channel (email | push | sms)
- category (marketing | updates | security)
- enabled (boolean)
- Unique constraint on [userId, channel, category]
```

You review the plan. The data model makes sense — a separate row per channel-category combination rather than a JSON blob. This is queryable, indexable, and clean. But you want one adjustment:

```
Good plan. One change: use PATCH instead of PUT — we want partial
updates so the client can toggle a single preference without sending all of them.
```

Claude Code acknowledges and adjusts. The plan is now locked in. Total time: about 2 minutes. You haven't written a line of code, but you've agreed on the data model, API design, and file locations. This prevents the expensive mistake of building something and then realizing the approach is wrong.

Phase 2: Tests First

With the plan agreed, you shift to test-driven development:

```
Write tests for notification preferences. Follow the patterns
in tests/api/user.test.ts. Cover:
- GET returns all preferences for authenticated user
- GET returns 401 for unauthenticated request
- PATCH updates a single preference
- PATCH validates channel and category values
```

```
- PATCH returns 401 for unauthenticated request
- PATCH returns 400 for invalid input
```

Claude Code reads `tests/api/user.test.ts` to understand your test conventions — the testing library (likely Vitest or Jest), how you set up test fixtures, how you mock authentication, and how you structure describe/it blocks. Then it creates `tests/api/notification-preferences.test.ts`.

The test file follows your existing patterns exactly: same import style, same helper functions for authenticated requests, same assertion patterns. It covers all six cases you specified.

You run the tests:

```
! npm test -- tests/api/notification-preferences.test.ts
```

All six tests fail. This is exactly what you want. Failing tests are your specification — they define precisely what the implementation must do. Each red test is a requirement waiting to be fulfilled.

```
6 tests, 0 passed, 6 failed
- NotificationPreference model not found (schema doesn't exist yet)
- Route /api/notifications/preferences not found (404)
```

Good. The failures confirm the tests are pointing at the right things. Move on.

Phase 3: Implementation

Now you build incrementally, starting from the foundation:

```
Implement starting with the Prisma schema. Add the NotificationPreference
model as we planned.
```

Claude Code edits `prisma/schema.prisma`, adding the model with the userId relation, the channel and category enums, the enabled boolean, and the unique constraint. Then it generates the migration:

```
npx prisma migrate dev --name add-notification-preferences
```

You run tests again:

```
! npm test -- tests/api/notification-preferences.test.ts
```

Two tests now have different errors — they can see the model but can't find the API route. Progress.

```
Now the API routes. Create GET and PATCH for /api/notifications/preferences.
Follow the auth middleware pattern in src/app/api/user/route.ts.
```

Claude Code reads the reference file, sees how authentication is handled (checking the session, returning 401 if missing), and creates `src/app/api/notifications/preferences/route.ts` with both handlers.

The GET handler queries all preferences for the authenticated user. If no preferences exist yet (new user), it returns defaults — all channels enabled for security, all disabled for marketing, a sensible default set.

The PATCH handler accepts a body with `channel`, `category`, and `enabled`, finds or creates the matching preference row, and updates it.

Run the tests:

```
! npm test -- tests/api/notification-preferences.test.ts
```

```
6 tests, 4 passed, 2 failed
```

Four passing. The two failures are the validation tests — the PATCH handler doesn't yet reject invalid channel or category values. It accepts anything and lets the database complain with an ugly error instead of a clean 400 response.

Phase 4: Iterate

This is where TDD pays off. The failing tests tell you exactly what's missing:

```
Add Zod validation to the PATCH handler. Reject invalid channel
and category values with a 400 response. Use the validation
pattern from src/lib/validations/user.ts.
```

Claude Code creates a Zod schema in `src/lib/validations/notification-preferences.ts`:

```
import { z } from "zod";

export const updatePreferenceSchema = z.object({
  channel: z.enum(["email", "push", "sms"]),
  category: z.enum(["marketing", "updates", "security"]),
  enabled: z.boolean(),
});
```

Then it updates the PATCH handler to validate the request body against this schema before processing, returning a 400 with specific error messages when validation fails.

Run the tests one more time:

```
! npm test -- tests/api/notification-preferences.test.ts
```

```
6 tests, 6 passed
```

All green. But you're not done. Run the full test suite to check for regressions:

```
! npm test
```

```
147 tests, 147 passed
```

Clean. Your new feature doesn't break anything existing.

One more thing — you notice an edge case the original test plan missed:

```
Add one more test: PATCH should return 404 if the user account
doesn't exist (deleted user with stale session). Add handling
in the route too.
```

Claude Code adds the test and the corresponding guard in the handler. Seven tests, seven passing. Full suite still green.

Phase 5: Polish & Commit

The feature is complete and tested. Time to ship:

```
Commit all notification preference changes with a descriptive message.
```

Claude Code stages the relevant files — the Prisma schema, migration, API route, validation schema, and test file — and creates a commit:

```
feat: add notification preferences API

Add GET/PATCH /api/notifications/preferences endpoints.
Users can toggle email/push/SMS per category (marketing,
updates, security). Includes Zod validation and full test
coverage.
```

The commit message follows your project's conventional commits format because Claude Code read your git history during the planning phase and matched the pattern.

Now create the pull request:

```
Create a PR targeting main. Summarize what we built.
```

Claude Code pushes the branch and creates the PR with a clear title and body describing the changes, the API contract, and the test coverage.

The entire feature — from planning to pull request — took about 15 minutes. A developer working alone might spend 45-90 minutes on the same feature. The time savings compound: build three features like this per day and you're saving 1.5 to 3.5 hours daily.

Lessons From This Workflow

Plan before you build. The 2-minute planning phase prevented wrong turns that would have cost 20 minutes to fix. Claude Code's plan was 90% right; your one adjustment (PUT to PATCH) caught a design

issue before any code existed.

Reference existing patterns. Every prompt pointed Claude Code at an existing file to follow. The tests matched your test conventions. The API routes matched your route conventions. The validation matched your validation conventions. The result reads like a team member wrote it, not an AI tool.

TDD caught edge cases. Writing tests first revealed that the PATCH handler needed input validation — something easy to forget when building implementation-first. The tests served as both a specification and a safety net.

Multiple small prompts beat one massive prompt. Five focused prompts produced better results than a single "build notification preferences with tests, validation, and a PR" mega-prompt would have. Each prompt had clear scope, and you could verify and adjust between steps.

The progression matters. Plan, then test, then implement, then iterate, then commit. This order works because each phase builds on the verified output of the previous phase. Skipping the plan leads to rework. Skipping tests leads to bugs. Skipping iteration leads to incomplete features.

This is the workflow that expert Claude Code users follow for every non-trivial feature. It feels slower at first — why plan when you could just start building? — but it's consistently faster end-to-end because you never build the wrong thing and you never ship untested code.

Chapter 18: Debugging Like a Pro

Most developers approach debugging with Claude Code backwards. They paste an error, say "fix this," and accept whatever Claude Code suggests. Sometimes this works. Often it produces a fix that addresses the symptom while leaving the root cause intact — or worse, introduces a new bug while silencing the original one.

Expert debugging with Claude Code follows a different pattern. It's methodical, evidence-driven, and produces fixes that actually solve the problem. This chapter teaches that pattern.

The Expert Debugging Mindset

The core principle is simple: **don't guess, investigate.** When something breaks, the natural instinct is to hypothesize immediately. "It's probably a null check." "I bet the database connection timed out." "Must be a CORS issue." Resist this instinct.

Instead, let Claude Code gather evidence before proposing anything. Claude Code can read files, search patterns, trace imports, check git history, and run commands — all faster than you can manually investigate. Your job is to point it at the problem and let it work.

The difference in practice:

Guessing approach:

```
The login is broken. Probably a session issue. Fix the session handling.
```

Claude Code modifies session handling based on your guess. If the guess is wrong, you've now changed working code and still have the original bug.

Investigation approach:

```
Login returns 500 since this morning. Here's the error:
! npm test -- --grep "login" 2>&1 | tail -20
```

Claude Code sees the actual error, reads the relevant files, traces the call stack, and finds the root cause — which might be session handling, or might be something completely different. The fix addresses what's actually broken.

The 5-Step Debugging Workflow

This workflow applies to every bug, from a one-line typo to a multi-service race condition. The steps scale with the complexity of the problem.

STEP 1: REPRODUCE

Before anything else, give Claude Code a way to see the failure. The best reproduction is a failing test or a command that triggers the error.

```
Getting "TypeError: Cannot read properties of undefined (reading 'email')"
when creating a new user. Reproduce:
! npm test -- tests/api/users.test.ts -t "create user" 2>&1 | tail -30
```

The `!` prefix runs the command inline and puts the output directly into your conversation. Claude Code sees the exact error, the stack trace, and the test context. The `2>&1` captures stderr (where most error

output goes), and `tail -30` keeps the output focused on the relevant part.

If you don't have a failing test, describe the steps:

```
Bug: clicking "Save" on the profile page shows a spinner forever.
No error in browser console. Network tab shows POST /api/profile
returns 200 but the response body is empty.
```

The reproduction tells Claude Code three things: what should happen, what actually happens, and where to start looking.

If you can't reproduce it, say so. "This happens intermittently — about 1 in 10 requests" is valuable information. Claude Code will look for race conditions, timing issues, or non-deterministic behavior rather than straightforward logic errors.

STEP 2: INVESTIGATE

This is the step most developers skip — and it's the most important one. Let Claude Code explore before it proposes anything.

```
Investigate the root cause. Read the relevant files, trace the call stack,
and check recent git changes. Don't propose a fix yet.
```

The explicit "don't propose a fix yet" is important. Without it, Claude Code might jump to a solution after reading one file. With it, Claude Code reads the handler, the service layer, the database query, checks the types, reviews recent commits, and builds a complete picture.

During investigation, Claude Code typically:

- Reads the file where the error originates
- Traces imports to find where data flows from
- Checks the database schema or type definitions
- Searches for similar patterns that work correctly
- Reviews recent commits that might have introduced the change
- Reads test files to understand expected behavior

Let this process run. The investigation might reveal that the "bug" is actually three bugs, or that the error message is misleading, or that the real problem is in a completely different file from where the error

surfaces.

STEP 3: HYPOTHESIZE

After investigation, Claude Code proposes a root cause. This is where you apply critical thinking. Don't accept the first hypothesis unchallenged.

```
What evidence supports that hypothesis? What would disprove it?
```

This question forces Claude Code to justify its reasoning with specific evidence from the codebase rather than general plausibility. A strong hypothesis looks like:

> "The error occurs because `getUserProfile()` in `src/services/user.ts` returns `null` when the user doesn't have a profile row, but the caller in `src/app/api/profile/route.ts` (line 23) destructures `.email` without a null check. This was introduced in commit `a3f2c1d` three days ago when the profile query was refactored — the old version returned a default object, the new version returns null."

A weak hypothesis looks like:

> "The error is probably because the user object is undefined somewhere. Let me add a null check."

The strong hypothesis identifies the exact file, line, commit, and mechanism. The weak one guesses. If Claude Code gives you a weak hypothesis, push back:

```
That's too vague. Which specific function returns undefined?
Show me the line where it breaks.
```

STEP 4: FIX MINIMALLY

Once the root cause is confirmed, apply the smallest possible fix. Not a refactor. Not "improvements while we're here." The minimal change that fixes the bug.

```
Fix it with the smallest change possible. Don't refactor anything else.
```

This constraint is essential. When Claude Code fixes a bug, it sometimes wants to “improve” surrounding code — rename variables, extract functions, add error handling to unrelated paths. Every additional change is a potential new bug and makes the fix harder to review.

A good bug fix changes 1-5 lines. It should be obvious from the diff what was wrong and what was fixed. If the fix requires changing 50 lines, either the root cause analysis was wrong or the problem is architectural — in which case, fix the immediate bug minimally and plan the refactor separately.

```
Fix the null check in the profile handler. If getUserProfile()
returns null, return a 404 response instead of destructuring.
```

Claude Code adds a guard clause — three lines of code. The fix is clear, reviewable, and precisely targeted.

STEP 5: VERIFY

Run the failing test to confirm the fix works, then run the full suite to check for regressions.

```
Run the failing test, then the full suite.
! npm test -- tests/api/users.test.ts -t "create user"
! npm test
```

Both must pass. If the targeted test passes but the full suite reveals a new failure, the fix is incomplete or has side effects. Go back to Step 2 and investigate the new failure.

Verification is non-negotiable. “It looks correct” is not verification. “All 147 tests pass” is verification.

Common Debugging Patterns

Different bugs call for different initial prompts. Here are the patterns that work best for common scenarios.

Runtime error with stack trace:

```
Getting this error in production. Here's the stack trace:
[paste the full stack trace]
Find the root cause — don't just add a try-catch around it.
```

The “don’t just add a try-catch” instruction prevents Claude Code from wrapping the problem instead of solving it.

Test passes locally, fails in CI:

```
This test passes on my machine but fails in CI. Here's the CI output:
[paste CI log]
Check for environment-dependent behavior: env vars, file paths,
timing assumptions, or dependency version differences.
```

Common causes: hardcoded paths, timezone assumptions, missing environment variables, different Node.js versions, test ordering dependencies.

Something changed and you don’t know what:

```
The search endpoint was working yesterday and returns empty results today.
! git log --oneline -10
! git diff HEAD~5 -- src/api/search.ts src/services/search.ts
```

Feeding the recent git history and diffs directly into context lets Claude Code see exactly what changed. Often the bug is visible in the diff.

Confusing or misleading error message:

```
Getting "ECONNREFUSED" on the payment endpoint but the payment
service is running. Read the error source code to understand
what actually triggers this error — the message might be misleading.
```

Some errors have generic messages that don’t describe the actual cause. Asking Claude Code to read the source of the error (the library code that throws it) often reveals the real trigger.

Performance issue:

```
The /api/reports endpoint takes 12 seconds for large date ranges.
Profile the database queries and identify which one is slow.
Check for missing indexes.
```

Be specific about what “slow” means — include the actual timing and the conditions that trigger it. “It’s slow” gives Claude Code nothing to work with. “12 seconds for date ranges over 90 days” gives it a clear investigation path.

The Systematic Debugging Skill

Claude Code includes a built-in skill for disciplined debugging:

```
/systematic-debugging
```

This skill enforces the 5-step workflow as a structured process. It prevents the common trap of jumping to fixes before completing investigation. The skill guides you through reproduction, evidence gathering, hypothesis formation, minimal fix application, and verification — with explicit checkpoints between each stage.

Use it when you catch yourself wanting to skip straight to “just fix it.” The skill adds about 2 minutes of discipline that saves 20 minutes of chasing wrong hypotheses.

Anti-Patterns That Make Debugging Worse

These habits are common and they all reduce debugging effectiveness.

“Fix it” without sharing the error. Claude Code can’t debug what it can’t see. Always include the error message, stack trace, or reproduction steps. If you don’t have them, your first prompt should be: “Help me reproduce this — the search feature sometimes returns wrong results but I don’t have a consistent repro.”

Guessing the cause in your prompt. “The bug is probably in the database query — fix it” sends Claude Code down a specific path that might be wrong. “The endpoint returns wrong results — investigate” lets Claude Code follow the evidence wherever it leads.

Accepting the first suggestion without evidence. Claude Code’s first hypothesis might be wrong. Before applying a fix, ask: “What evidence from the codebase supports this diagnosis?” If Claude Code can’t point to specific lines, the hypothesis is a guess.

Adding try-catch everywhere. This is the debugging equivalent of putting tape over your car’s warning light. The error goes away — but the problem remains, now silently. If Claude Code suggests wrapping something in try-catch, ask: “What error are we catching and why does it occur? Can we prevent it instead of catching it?”

Adding logging "just in case." Scattering `console.log` statements is a legacy debugging technique from before AI-assisted development. Claude Code can read the code and trace the data flow without executing it. Logging makes sense when you need runtime values that can't be inferred from code reading — but for most bugs, Claude Code can identify the issue through static analysis alone.

Fixing the symptom instead of the cause. If a function returns `undefined` and you add a fallback default, the function still returns `undefined` — you've just hidden it. Fix why it returns `undefined`. The extra 5 minutes of investigation saves hours of cascading bugs downstream.

The developers who get the most value from Claude Code during debugging are the ones who treat it as an investigator rather than a fixer. Point it at the problem, let it gather evidence, challenge its hypothesis, and only then apply the minimal fix. This approach is slower for trivial bugs (where "fix it" would have worked) but dramatically faster for everything else — and it never makes things worse.

Chapter 19: Cost Optimization & Efficiency

Claude Code is powerful, but it's not free. Whether you're on a subscription plan or paying per token via API key, understanding how costs work — and how to minimize them without reducing quality — is a practical skill that pays for itself immediately.

This chapter gives you the concrete strategies that experienced users rely on to keep costs low while maintaining high output. No vague advice — specific techniques with measurable impact.

Understanding Token Costs

Every interaction with Claude Code consumes tokens. A token is roughly 3-4 characters of English text. Your costs come from two categories:

Input tokens — everything Claude Code reads: your prompts, file contents, CLAUDE.md instructions, conversation history, tool outputs, and MCP server responses. This is the context that Claude Code uses to understand your request.

Output tokens — everything Claude Code produces: responses, code it writes, commands it runs, tool calls it makes. Output tokens are more expensive than input tokens, typically 3-5x the cost per token.

This ratio matters. Reading a 500-line file (input) costs far less than Claude Code writing a 500-line file from scratch (output). Strategies that reduce output tokens save more than strategies that reduce input tokens by the same amount.

Track your spending with built-in commands:

```
/cost
```

This shows total tokens consumed and estimated cost for the current session. Check it periodically during long sessions — costs can accumulate faster than you expect when Claude Code is reading many files or generating long outputs.

For hard limits, set a budget cap when starting a session:

```
claude --max-budget-usd 5.00
```

The session ends gracefully when the budget is reached. Set low budgets for exploration sessions (browsing code, asking questions) and higher budgets for implementation sessions (building features, running tests).

The 80/20 of Cost Reduction

Five techniques account for the vast majority of cost savings. Master these first.

1. Read less: use offset and limit.

When you know which part of a file you need, tell Claude Code exactly where to look. Reading a 2,000-line file when you only need lines 150-200 wastes tokens on 1,950 irrelevant lines.

```
Read lines 150-200 of src/services/payment.ts — the refund logic.
```

Claude Code reads 50 lines instead of 2,000. If you do this ten times per session across different files, the savings are substantial.

2. Edit, don't Write.

The Edit tool sends only the diff — the old text being replaced and the new text replacing it. The Write tool sends the entire file contents. For a 400-line file where you're changing 5 lines, Edit transmits roughly 15 lines while Write transmits all 400.

Reinforce this in your CLAUDE.md:

```
- Always use Edit for modifying existing files
- Only use Write for creating new files
```

This single rule can cut output token costs by 30-50% for sessions heavy on file modifications.

3. Compact proactively.

The `/compact` command summarizes your conversation history into key points and frees up context space. Don't wait for auto-compression — it triggers around 60% context utilization and Claude Code decides what to keep.

```
/compact focus on the API changes and test results
```

The focus argument tells Claude Code what matters most. Without it, Claude Code might discard the exact context you need for your next prompt. Compact every 15-20 exchanges or whenever you notice the conversation getting long.

4. Start fresh conversations.

Long conversations carry all their history as input tokens. Every prompt in a 50-exchange conversation pays for the accumulated context of the previous 49 exchanges (compressed, but still substantial).

When you finish one task and start another, begin a new conversation. Don't reuse a debugging session for feature development. Don't keep a morning session alive into the afternoon. Fresh conversations have minimal context overhead.

```
# Finished debugging? Start fresh for the next task.
claude
```

5. Choose the right model for the task.

Not every task needs the most powerful model. Claude Code supports model switching mid-session with `Shift+Tab`. Match the model to the task complexity.

Model Selection Strategy

Task	Recommended Model	Why
“What does this function do?”	Haiku	Simple comprehension, fast response
“Add a CRUD endpoint”	Sonnet	Standard coding, good balance
“Refactor the auth architecture”	Opus	Complex reasoning, many files
“Fix this typo”	Haiku	Trivial change, minimal reasoning
“Debug a race condition”	Opus	Deep reasoning, subtle causation
“Write unit tests”	Sonnet	Pattern-following, moderate complexity
“Rename a variable across files”	Haiku	Mechanical, no judgment needed
“Design the database schema”	Opus	Architectural decisions, trade-offs

The cost difference is significant. Haiku costs roughly 1/60th of Opus per token. A 10-minute session with Haiku for quick questions costs what 10 seconds of Opus output costs. Using Opus to fix a typo is like hiring an architect to change a lightbulb.

The practical workflow: start your session with Sonnet (the default). Switch to Haiku with `Shift+Tab` for quick lookups and simple changes. Switch to Opus for the hard problems — architecture decisions, complex debugging, multi-file refactors. Switch back to Sonnet when the hard part is done.

Prompt Efficiency

The way you write prompts directly affects token consumption. Vague prompts cause back-and-forth exchanges (each one consuming tokens). Specific prompts get results in one shot.

Expensive approach (3 exchanges):

```
Exchange 1: "Fix the bug in the user service."
Exchange 2: "The one where it returns null instead of throwing."
Exchange 3: "In the getUserById function, line 45."
```

Cheap approach (1 exchange):

```
"Fix getUserById in src/services/user.ts (line 45) — it returns null
instead of throwing NotFoundError when the user doesn't exist."
```

The single specific prompt costs roughly one-third of the three-exchange version because it avoids two rounds of input context being re-sent.

Include these elements when possible: - **File paths** — eliminates searching - **Line numbers** — eliminates scrolling - **Expected vs actual behavior** — eliminates guessing - **Reference files** — eliminates asking for clarification

Every clarifying question Claude Code asks is a round trip that doubles the cost of that interaction. Front-load the context and eliminate the questions.

Subscription vs API Key

Claude Code offers two pricing approaches. Understanding when each makes sense prevents overpaying.

Subscription plans: - **Pro ($20/month):** Includes a daily message allowance. Best for moderate, consistent usage — a few tasks per day. Messages reset daily. If you hit the limit, you wait or upgrade. - **Max ($100-200/month):** Higher daily limits, priority access to powerful models. Best for professional developers who use Claude Code as their primary coding tool throughout the day.

API key (pay-per-token): - No message limits — pay exactly for what you use - Costs vary by model: Haiku is very cheap, Opus is premium - No daily resets — use as much or as little as needed - Best for variable usage patterns or heavy burst usage

Rule of thumb: Track your API spending for a month. If you're consistently spending over $100/month on API tokens, the Max plan likely saves money. If you spend $20-50/month, Pro might be sufficient. If you use Claude Code sporadically (a few sessions per week), API key is cheapest because you only pay for what you use.

You can check subscription usage with:

```
/stats
```

This shows session counts, token consumption, and usage patterns. Review it weekly to understand whether your plan matches your actual usage.

Hidden Cost Traps

Certain patterns silently inflate your token costs. Watch for these.

Large files in context. `package-lock.json` can be 50,000+ lines. If Claude Code reads it (or if it's accidentally included via a broad search), you've consumed a massive number of input tokens for minimal value. Use `.claudeignore` to exclude generated files:

```
# .claudeignore
package-lock.json
yarn.lock
pnpm-lock.yaml
*.min.js
*.min.css
dist/
build/
coverage/
node_modules/
```

This single file can cut per-session costs by 10-20% depending on your project structure.

Re-reading the same files. In a long conversation, Claude Code might read the same file multiple times as earlier reads get compressed out of context. If you know you'll reference a file repeatedly, mention it once at the start and use `/compact` with instructions to preserve its contents.

Write instead of Edit. Already covered, but worth repeating because it's the single most common cost waste. Every time Claude Code uses Write to modify an existing file, you pay for the entire file as output tokens. A 300-line file modified via Write costs 60x more in output tokens than the same change via Edit (5 lines of diff).

Overly broad searches. "Search the entire codebase for anything related to authentication" might read hundreds of files. "Search `src/auth/` for the session validation logic" reads three. Guide Claude Code to the right directory and it won't need to search broadly.

Long conversations without compaction. After 30+ exchanges, your conversation history might consume 50,000+ tokens of input on every single prompt. That's 50,000 tokens of context being re-processed every time you type a one-line request. Use `/compact` or start fresh.

Unnecessary code generation. If Claude Code offers to generate boilerplate that your framework provides (scaffolding commands, template generators), ask it to use the framework's generator instead. `npx prisma generate` produces the Prisma client in milliseconds and zero output tokens. Having Claude Code write the equivalent code manually costs tokens and produces an inferior result.

Cost-Effective Workflows

Combining these techniques into daily habits produces consistent savings.

Morning startup: Fresh conversation. Sonnet model. State the day's goal clearly. Include file paths.

Quick questions mid-task: Switch to Haiku. Ask the question. Switch back.

Complex implementation: Stay on Sonnet. Reference existing patterns. Use Edit not Write. Compact every 20 exchanges.

Architecture decisions: Switch to Opus for the planning phase. Get the plan approved. Switch to Sonnet for implementation.

End of day: Check `/cost`. Review where tokens went. Identify any wasteful patterns to avoid tomorrow.

The goal isn't to minimize spending at the expense of productivity. A $5 session that saves you 3 hours of work is excellent value. The goal is to stop spending $5 on work that should cost $0.50 — by being specific, staying focused, and matching the model to the task.

Chapter 20: Your Custom Setup Blueprint

You've learned the individual pieces — CLAUDE.md, memory, hooks, MCP servers, skills, permissions, prompt patterns, debugging workflows, cost management. This final chapter ties them all together into a complete, personalized setup that makes Claude Code feel less like a tool and more like a team member who knows your codebase, your preferences, and your workflows.

The difference between a beginner and an expert Claude Code user isn't knowledge. It's configuration. The expert configured their environment once and benefits from it every single session. This chapter gives you the blueprint to do the same.

The Complete Setup Checklist

Work through this list over the coming weeks. You don't need to do everything at once — each item adds value independently.

- ☐ Global `CLAUDE.md` with your personal preferences and coding style
- ☐ Project `CLAUDE.md` with project-specific rules, conventions, and architecture notes
- ☐ Permission allowlists for commands you run frequently (`npm test`, `git commit`, etc.)
- ☐ `.claudeignore` to exclude generated files, lock files, and build artifacts
- ☐ PostToolUse hook for automatic code formatting on every edit
- ☐ At least one MCP server connected (database, GitHub, or project-specific)
- ☐ Custom keybindings for your most-used actions
- ☐ Auto-memory enabled for persistent context across sessions
- ☐ One custom skill for your most repeated workflow
- ☐ Three-layer settings hierarchy configured (global, project, local)

Each checked box removes friction from your daily workflow. Permissions stop interrupting you. Formatting happens automatically. Memory means you never re-explain your project. The compound effect is substantial.

Starter Global CLAUDE.md Template

This file lives at `~/.claude/CLAUDE.md` and applies to every project you open. Keep it focused on universal preferences — language, style, and behavior that don't change between projects.

```
# Global Preferences

## Coding Style
- Use TypeScript strict mode. No `any` types.
- Prefer `const` over `let`. Never use `var`.
- Use early returns over nested if-else.
```

```
- Descriptive variable names. No single-letter variables except loop counters.

## Behavior
- Always use Edit for modifying existing files, Write only for new files.
- Run tests after making changes. Don't tell me it "should work."
- When committing, use conventional commits (feat:, fix:, refactor:, etc.).
- Keep explanations brief. I'll ask if I need more detail.

## Don't
- Don't add comments explaining obvious code.
- Don't refactor unrelated code when fixing a bug.
- Don't use console.log for debugging — use proper error handling.
- Don't create documentation files unless I ask for them.
```

This template is 15 lines of actual rules. Every rule is actionable and specific. “Use early returns over nested if-else” changes how Claude Code writes every function. “Always use Edit for modifying existing files” saves tokens on every file change. These rules compound across thousands of interactions.

Customize this template to match your real preferences. If you prefer Python, replace the TypeScript rules. If you like verbose explanations, remove the brevity rule. The template is a starting point, not a prescription.

Starter Settings.json Template

This file lives at `.claude/settings.json` in your project root. Commit it to git so your team shares the same configuration.

```
{
  "permissions": {
    "allow": [
      "Read",
      "Glob",
      "Grep",
      "Bash(npm test *)",
      "Bash(npm run lint *)",
      "Bash(npm run build *)",
      "Bash(npx prisma *)",
      "Bash(git (add|commit|status|diff|log|push|branch) *)"
    ]
  },
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
```

```
            "type": "command",
            "command": "prettier --write $CLAUDE_FILE_PATH 2>/dev/null || true"
          }
        ]
      }
    ]
  },
  "mcpServers": {
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres"],
      "env": {
        "DATABASE_URL": "${DATABASE_URL}"
      }
    }
  }
}
```

This template does three things. The permission allowlist auto-approves safe commands — reading files, running tests, building, and standard git operations — so Claude Code doesn't pause for approval on routine actions. The PostToolUse hook runs Prettier on every file Claude Code edits, ensuring consistent formatting without you thinking about it. The MCP server connects Claude Code to your PostgreSQL database so it can inspect schemas, run queries, and understand your data model directly.

Replace `prettier` with your formatter (`black`, `gofmt`, `rustfmt`). Replace `postgres` with your database. Adjust the git pattern if you want tighter restrictions (removing `push`, for example, to require manual approval for pushes).

The `|| true` after the prettier command prevents the hook from blocking Claude Code if formatting fails (for example, on a non-code file). Without it, a formatter error on a markdown file would halt the session.

Recommended MCP Servers by Stack

MCP servers extend what Claude Code can access. The right servers depend on your technology stack.

Web development (Next.js, React, Node.js): - **GitHub MCP** — Create PRs, manage issues, review code, check CI status without leaving Claude Code - **PostgreSQL/MySQL MCP** — Query your database, inspect schemas, understand data relationships - **Puppeteer MCP** — Browser automation for testing UI flows, taking screenshots, verifying deployments

Mobile development (iOS, Android, React Native): - **GitHub MCP** — Same benefits as web - **Firebase MCP** — Manage Firestore collections, check authentication setup, review security rules - **SQLite MCP** — Inspect local databases, debug data persistence issues

Data science and ML: - **SQLite MCP** — Query local datasets, inspect feature stores - **Filesystem MCP** — Navigate large data directories, manage experiment outputs - **Custom analysis MCP** — Wrap your analysis scripts as MCP tools for repeatable investigations

DevOps and infrastructure: - **GitHub MCP** — Monitor deployments, review infrastructure PRs - **Docker MCP** — Manage containers, inspect logs, debug service connectivity - **AWS/GCP/Azure MCP** — Check cloud resources, review configurations, monitor costs

Start with one MCP server — whichever addresses your biggest daily friction point. GitHub is the most universally useful. Add more as you discover needs.

Building Your Setup Over Time

Don't configure everything in one sitting. Build your setup progressively, validating each addition before layering the next.

Week 1: Foundation. Create your global `CLAUDE.md` with 5-10 personal rules. Create a project `CLAUDE.md` with your project's key conventions. Add a basic permission allowlist for read operations and test commands. This alone transforms the experience — Claude Code follows your style and stops interrupting for routine approvals.

Week 2: Formatting hook. Add the PostToolUse hook for your formatter. From this point on, every file Claude Code touches is automatically formatted. No more inconsistent code style. No more manual formatting passes. This is the single highest-ROI configuration you can add.

Week 3: MCP servers. Connect your primary database and GitHub. Claude Code can now query your data model directly (no more "what columns does the users table have?") and manage PRs without leaving the terminal. These two connections eliminate the most common context-switching interruptions.

Week 4: Keybindings and workflow optimization. Customize keybindings for actions you repeat often. Review your first month's usage — which prompts do you type repeatedly? Which tasks follow the same pattern? Identify candidates for skills.

Month 2: First custom skill. Take your most repeated workflow — deployment, PR review, feature scaffolding — and encode it as a skill. Use it for two weeks. Refine based on experience. A single well-built skill saves 10-20 minutes per use.

Month 3: Advanced automation. Set up scheduled agents for recurring tasks (dependency updates, daily PR reviews). Add path-specific rules for different parts of your codebase. Connect additional MCP servers. At this point, your setup is mature and Claude Code operates with deep knowledge of your project, your tools, and your preferences.

Before and After

The difference between an unconfigured and a configured Claude Code setup is stark. Here's what changes.

Before — out of the box: - Claude Code asks permission for every `npm test`, every `git status`, every file read - Code it writes uses its own style conventions, not yours - Every new session starts from zero — no memory of your project, your preferences, or previous decisions - You manually format code after every edit - You switch to a browser for GitHub operations, database queries, and documentation lookups - Generic responses that don't account for your project's architecture or conventions

After — fully configured: - Routine commands run automatically; permissions only pause for genuinely dangerous operations - Every line of code matches your team's style because CLAUDE.md defines it and hooks enforce it - Memory persists across sessions — Claude Code remembers your architecture decisions, naming conventions, ongoing work, and known issues - Code is auto-formatted the instant Claude Code saves a file - GitHub PRs, database queries, and external tools are accessible directly from the conversation - Responses are project-aware: Claude Code knows your framework, your patterns, your testing conventions, and your deployment process - One-keypress commits, one-command deployments, automated quality checks

The unconfigured experience feels like working with a capable stranger. The configured experience feels like working with a team member who has read your entire codebase, attended every architecture meeting, and remembers everything.

The Compound Effect

Each individual configuration saves a small amount of time. Permission allowlist: 2 seconds per approval, maybe 50 approvals per day — less than 2 minutes saved. Formatting hook: 5 seconds per manual format, 30 times per day — 2.5 minutes. Memory: 1 minute re-explaining context per session, 5 sessions per day — 5 minutes.

Individually, these feel insignificant. Combined, they save 30-45 minutes per day. Over a month, that's 10-15 hours. Over a year, it's 25-35 full working days. And the quality improvement compounds too — consistent formatting, persistent memory, and project-aware responses mean fewer bugs, less rework, and cleaner code.

The developers who report the highest satisfaction with Claude Code aren't the ones using the most advanced features. They're the ones with the best configurations. They spent 2 hours setting things up and now save 30 minutes every day, forever.

Your Next Step

Close this book. Open your terminal. Create `~/.claude/CLAUDE.md` with five rules that matter to you. That's it — five rules. Use Claude Code for a day with those rules active and notice the difference.

Then add a permission allowlist. Then a formatting hook. Then a project CLAUDE.md. Each addition takes 5 minutes and improves every session that follows.

You now have everything you need to use Claude Code at an expert level. The knowledge is in the previous 19 chapters. The execution starts with one file, five rules, and the decision to configure your tools instead of accepting their defaults.

The difference between a developer and a 10x developer has never been typing speed. It's tooling, automation, and the discipline to invest 10 minutes of setup for 10 hours of payoff.

Configure once. Benefit forever.